



ESCUELA POLITÉCNICA



UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

GRADO EN INGENIERÍA INFORMÁTICA DE COMPUTADORES

TRABAJO FIN DE GRADO

**Generación de conjuntos de datos en el seguimiento automático de
personas por robots**



ESCUELA POLITÉCNICA



UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

GRADO EN INGENIERÍA INFORMÁTICA DE COMPUTADORES

TRABAJO FIN DE GRADO

**Generación de conjuntos de datos en el seguimiento automático de
personas por robots**

Autor: Alejandro González Fernández

Tutor: Pablo Bustos García de Castro

Co-Tutor: Alejandro Torrejón Harto

Índice general

1. Introducción	5
2. Objetivos	7
3. Solución Propuesta	9
3.1. Fase 1. Preparación del entorno	11
3.1.1. Requisitos del Sistema	11
3.1.2. Herramientas y Tecnologías Utilizadas	13
3.1.3. Configuración del Control de Versiones	14
3.1.4. Instalación y Configuración	16
3.2. Fase 2. Generación de archivo de vídeo RGBD	28
3.2.1. Introducción	28
3.2.2. Objetivos	29
3.2.3. Explicaciones técnicas	30
3.2.4. Librerías principales	32
3.2.5. Desarrollo del código	34
3.2.6. Resultados	37
3.2.7. Conclusiones	38
3.2.8. Ampliaciones a futuro	39
3.3. Fase 3. Generación y clasificación de dataset	41
3.3.1. Introducción	41
3.3.2. Objetivos	43
3.3.3. Explicaciones técnicas	44

3.3.4.	Librerías principales	46
3.3.5.	Desarrollo del código	47
3.3.6.	Resultados	54
3.3.7.	Conclusiones	57
3.3.8.	Ampliaciones a futuro	58
3.4.	Fase 4. Entrenamiento de la red neuronal	59
3.4.1.	Introducción	59
3.4.2.	Objetivos	60
3.4.3.	Explicaciones técnicas	61
3.4.4.	Librerías principales	63
3.4.5.	Desarrollo del código	64
3.4.6.	Resultados	69
3.4.7.	Conclusiones	71
3.4.8.	Ampliaciones a futuro	72
4.	Implementación del sistema final	75
4.1.	Introducción	75
4.2.	Objetivos	76
4.3.	Explicaciones técnicas	77
4.4.	Librerías principales	78
4.5.	Desarrollo del código	80
4.5.1.	Detección sobre vídeo pregrabado	80
4.5.2.	Detección sobre video en tiempo real	83
4.6.	Resultados	85
4.7.	Conclusiones	86
4.8.	Ampliaciones a futuro	87
5.	Resultados	89
6.	Conclusiones	91

7. Trabajos Futuros	93
Bibliografía	95

Índice de tablas

Índice de figuras

3.1. Generación USB Bootable.	17
3.2. Selección de arranque de USB Bootable.	18
3.3. Menu de selección del tipo de instalación.	19
3.4. Pagina web de VS Code.	20
3.5. Extensión Python de VS Code.	20
3.6. Extensión Error Lens de VS Code.	21
3.7. Extensión Autodocstring de VS Code.	21
3.8. Modificación del PATH para añadir las herramientas.	26
3.9. Interfaz de usuario fase 2.	36
3.10. Información del archivo de grabación.	38
3.11. Ejemplo del archivo JSON generado.	38
3.12. Formato carpetas de fotogramas.	55
3.13. Ejemplo del archivo JSON generado.	55
3.14. Formato carpetas de ROIs.	56
3.15. Ejemplo del archivo JSON generado.	57
3.16. Archivo que guarda los parámetros entrenados de la red.	69
3.17. Gráfico de los resultados de entrenamiento y validación.	70
3.18. Ejemplo del archivo JSON generado.	71
4.1. Interfaz de usuario con la imagen original y otra con los resultados. . .	86

Presentación

Soy Alejandro González Fernández, estudiante de Grado en Ingeniería Informática de Computadores con especialización en Inteligencia Artificial en la Universidad de Extremadura.

Mi proyecto de fin de grado se centra en la aplicación de la Inteligencia Artificial en la vida cotidiana. El objetivo principal es unificar la inteligencia artificial y la robótica para resolver tareas comunes de manera eficiente. En el proceso, se diseñan una serie de códigos que permiten a los usuarios generar conjuntos de datos clasificados a partir de videos, entrenar redes neuronales para su implementación práctica en diversas aplicaciones de la vida real.

Los resultados de este proyecto subrayan el potencial de la inteligencia artificial para transformar nuestro día a día, optimizando procesos y facilitando la interacción hombre-máquina de manera más intuitiva y eficaz.

Agradecimientos

Quisiera expresar mi más profundo agradecimiento a todas las personas que han contribuido al desarrollo de este Trabajo de Fin de Grado. En primer lugar, deseo agradecer de manera muy especial a mis tutores, Pablo Bustos y Alejandro Torrejón. Su guía experta, paciencia y continuo apoyo han sido fundamentales para la realización de este proyecto. Su habilidad para ofrecer orientación clara y motivación constante ha hecho que cada desafío se convierta en una oportunidad de aprendizaje.

También quiero extender mi agradecimiento a todo el equipo de RoboLab[1], un departamento dedicado a la investigación de nuevas tecnologías aplicadas a la realidad virtual. A todos los investigadores, técnicos y compañeros que trabajan en esta sección, gracias por su invaluable ayuda y colaboración. Su disposición para resolver problemas y compartir sus conocimientos ha sido crucial para superar los obstáculos encontrados a lo largo del proyecto.

A todos, por su tiempo, por sus consejos y por el apoyo brindado, les estoy inmensamente agradecido. Este trabajo no habría sido posible sin su contribución y dedicación.

Capítulo 1

Introducción

La inteligencia artificial (IA) ha transformado diversas áreas de la vida cotidiana, ofreciendo soluciones innovadoras que mejoran la eficiencia y precisión en tareas anteriormente realizadas por humanos. En particular, el uso de técnicas de IA para la detección y seguimiento de personas ha cobrado una relevancia significativa en múltiples aplicaciones, como la seguridad, la videovigilancia, y la interacción humano-robot. Este Trabajo de Fin de Grado (TFG) se enfoca en la implementación y evaluación de un sistema de seguimiento basado en inteligencia artificial, cuyo objetivo es identificar y seguir a una persona específica en entornos dinámicos, distinguiéndola de otras personas en la escena.

El seguimiento de personas en tiempo real representa un desafío considerable debido a las variaciones en la apariencia, la oclusión, los cambios en la iluminación, y la presencia de múltiples individuos en el mismo entorno. Para abordar estos desafíos, este proyecto emplea redes neuronales profundas, que han demostrado ser efectivas en la tarea de detección y reconocimiento de objetos y personas. El enfoque particular de este TFG es desarrollar un sistema que no solo identifique a las personas presentes en un escenario, sino que también discrimine con precisión al objetivo específico del seguimiento, incluso cuando este se encuentra entre otros individuos.

La implementación de este sistema se fundamenta en la combinación de técnicas de visión por computadora y modelos de aprendizaje profundo, aprovechando tanto

la capacidad de las redes neuronales para procesar imágenes en tiempo real como su habilidad para aprender y mejorar a partir de grandes cantidades de datos. Este trabajo se enfoca en crear un sistema funcional y eficiente. En última instancia, el proyecto pretende contribuir al desarrollo de soluciones más robustas y precisas, que puedan ser aplicadas en una variedad de contextos prácticos, desde la robótica hasta la vigilancia automatizada, mejorando la interacción entre sistemas automatizados y los seres humanos en escenarios cotidianos.

Capítulo 2

Objetivos

El objetivo principal de este Trabajo de Fin de Grado es diseñar e implementar un sistema completo para el seguimiento de una persona específica en entornos dinámicos, utilizando una combinación de grabaciones de cámara RGB-D, generación de datasets personalizados, y redes neuronales profundas. El sistema final deberá ser capaz de identificar y seguir a la persona objetivo, distinguiéndola de otros individuos en la escena, con alta precisión y fiabilidad.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

- ★ **Captura y almacenamiento de datos RGB-D:** Desarrollar un módulo que permita la captura de grabaciones mediante una cámara RGB-D (color y profundidad) y su almacenamiento eficiente en disco. Este paso es fundamental para asegurar la disponibilidad de datos de alta calidad para las siguientes fases del proyecto.
- ★ **Generación de un dataset personalizado:** Utilizar las grabaciones capturadas para crear un dataset estructurado que sirva de base para el entrenamiento de la red neuronal. Este dataset deberá estar compuesto por imágenes etiquetadas que contengan tanto a la persona objetivo como a otros individuos, facilitando la tarea de discriminación durante el entrenamiento.
- ★ **Diseño y entrenamiento de una red neuronal:** Crear una arquitectura de

red neuronal adecuada para la tarea de detección y seguimiento de personas en entornos dinámicos. El modelo deberá ser entrenado utilizando el dataset generado, optimizando sus parámetros para maximizar la precisión en la identificación del objetivo.

- ★ **Desarrollo de un sistema de seguimiento en tiempo real:** Implementar un programa final que integre la red neuronal entrenada y permita el seguimiento en tiempo real de la persona objetivo. Este sistema deberá ser capaz de procesar nuevas grabaciones de la cámara RGB-D, aplicar el modelo de seguimiento, y mostrar al usuario el resultado, destacando al objetivo sobre los demás individuos en la escena.
- ★ **Optimización del equilibrio entre eficiencia y legibilidad:** A lo largo del desarrollo del sistema, se priorizará encontrar un equilibrio óptimo entre la eficiencia del código y la legibilidad del mismo. Esto implicará implementar soluciones que no solo sean rápidas y efectivas en términos de procesamiento de datos y ejecución en tiempo real, sino que también mantengan un código claro y fácilmente comprensible. Se buscará que el sistema sea modular, bien documentado y fácil de mantener, de manera que futuras mejoras o adaptaciones puedan realizarse sin comprometer el rendimiento ni la claridad del código.

Estos objetivos permitirán no solo el desarrollo de un sistema funcional de seguimiento de personas, sino también la exploración de técnicas avanzadas en el manejo de datos de cámaras RGB-D, la creación de datasets, y el uso de redes neuronales en aplicaciones prácticas de inteligencia artificial.

Capítulo 3

Solución Propuesta

El proyecto se ha desarrollado de manera modular, permitiendo que cada componente funcione de forma independiente pero integrada. Esta modularidad facilita la incorporación de nuevas funcionalidades de manera simple y eficiente, lo que significa que cualquier usuario que desee expandir, mejorar o adaptar el proyecto puede hacerlo sin necesidad de reestructurar completamente el código existente.

Además, se ha comentado exhaustivamente todo el código, con el objetivo de que cualquier usuario pueda entender su funcionamiento y uso sin dificultad. Para mayor comodidad, se han creado scripts que automatizan la ejecución de los componentes de RoboComp, así como la descarga de las librerías necesarias para la ejecución del proyecto. Estos scripts simplifican significativamente el proceso de configuración y uso del proyecto, ahorrando tiempo y reduciendo la posibilidad de errores en la instalación.

La solución propuesta se estructura en torno a cuatro fases clave, las cuales son fundamentales para garantizar el éxito de la implementación del proyecto. Cada una de estas fases se describe en detalle en este capítulo, y juntas, forman la base para la fase final, que es la implementación efectiva del proyecto. Estas fases no solo facilitan la preparación y planificación adecuada, sino que también aseguran que cada aspecto del proyecto esté cuidadosamente considerado y ejecutado. A continuación, se describen brevemente las cuatro fases, que son esenciales para el logro de los objetivos propuestos:

- ★ **Fase 1. Preparación del entorno:** Esta fase inicial se centra en la preparación del entorno de desarrollo. Incluye la especificación y configuración de las herramientas necesarias, la instalación del sistema operativo adecuado, y la configuración de las librerías básicas y comunes que serán utilizadas a lo largo del proyecto. La correcta preparación de este entorno técnico es crucial, ya que sienta las bases sobre las cuales se desarrollará el proyecto.
- ★ **Fase 2. Generación de archivo de vídeo RGBD:** En esta fase, se desarrolla el código necesario para establecer una conexión vía USB con una cámara RGBD. Este código permitirá la transmisión de datos en tiempo real desde la cámara hacia el sistema, con el objetivo de realizar grabaciones de vídeo utilizando esos datos y almacenarlas en disco.
- ★ **Fase 3. Generación y clasificación de dataset:** Esta fase se centra en la creación de un dataset clasificado basado en los vídeos obtenidos en la fase anterior. Para diversificar el trabajo y optimizar el proceso, se desarrollan dos códigos específicos:
 - ★ **Primer código:** Se encarga de extraer y guardar los fotogramas del vídeo en disco, generando un conjunto de imágenes base para el dataset.
 - ★ **Segundo código:** Clasifica las Regiones de Interés (ROIs) que contienen personas en las imágenes almacenadas, separándolas entre “persona objetivo” y “no objetivo”. Se intentará automatizar este proceso lo más posible, para aumentar la eficiencia y reducir la intervención manual.

Esta fase es esencial para construir un dataset robusto y bien clasificado, que será utilizado en las etapas posteriores del proyecto.

- ★ **Fase 4. Entrenamiento de la red neuronal:** En esta última fase, se utiliza el dataset clasificado generado en la fase anterior para entrenar una red neuronal. Esta red neuronal se encargará de identificar a la persona objetivo entre múltiples personas presentes en un fotograma. Por ejemplo, si en un fotograma hay cuatro

personas, la red debe ser capaz de identificar, si está presente, a la persona objetivo entre ellas. Este entrenamiento es crucial para que la red aprenda a reconocer características distintivas de la persona objetivo y pueda realizar la identificación con alta precisión en escenarios reales.

3.1. Fase 1. Preparación del entorno

3.1.1. Requisitos del Sistema

En cuanto a hardware, no es necesario disponer de un procesador específico; para este proyecto se ha utilizado lo siguiente:

- ★ **Procesador:** Intel Core i7 de 8ª generación.
- ★ **RAM:** Es recomendable contar con al menos 8 GB aunque se han utilizado 16 GB.
- ★ **Memoria:** Es imprescindible que se utilicen memorias con una alta capacidad debido al tamaño considerable de los vídeos. Por esto se ha contado con un SSD de 1TB al que adicionalmente se le ha añadido un disco duro externo de 1 TB extra.
- ★ **Tarjeta gráfica:** Será un componente importante ya que disponer de “GPU” potente mejora los tiempos de entrenamiento y uso de redes neuronales.
- ★ **Periféricos:** Se recomienda disponer de un ratón y teclado inalámbrico ya que el “NUC” (Next Unit of Computing) se encuentra anclado al robot y en ocasiones podría estar moviéndose.
- ★ **Cámara RGBD:** Es un elemento clave en el desarrollo del proyecto, específicamente es una cámara de Intel cuyo modelo es D455.

En cuanto al sistema operativo, se ha optado por utilizar “Ubuntu 22.04 LTS” debido a su estabilidad y soporte extendido, proporcionando un entorno robusto para el desarrollo de aplicaciones complejas.



3.1. FASE 1. PREPARACIÓN DEL ENTORNO

El lenguaje de programación principal es “Python 3.10.12”, seleccionado por sus últimas características y mejoras que facilitan las diversas fases del proyecto, especialmente en el procesamiento de datos, implementación de algoritmos de inteligencia artificial y su integración con frameworks modernos.

Para el desarrollo de código, se ha elegido Visual Studio Code en su última versión. Este entorno destaca por su versatilidad y amplia gama de extensiones que facilitan el desarrollo en múltiples lenguajes de programación. Las extensiones específicas utilizadas están diseñadas para optimizar el flujo de trabajo y mejorar la productividad durante el desarrollo.

El proyecto hace uso intensivo del framework RoboComp para la creación de componentes con interfaces asociadas, permitiendo la implementación de tareas complejas de forma modular. Además, se emplean bibliotecas esenciales como NumPy y OpenCV para operaciones numéricas y procesamiento de imágenes y videos, respectivamente. PyTorch se utiliza para el desarrollo y entrenamiento de redes neuronales, mientras que librerías específicas como pyrealsense2 y YOLOv8s complementan el flujo de trabajo con funcionalidades especializadas en visión computacional y detección de objetos.

Git se utiliza como sistema de control de versiones, con un flujo de trabajo organizado en ramas dedicadas a cada fase del proyecto. Las ramas de desarrollo permiten trabajar de manera paralela en diferentes componentes y funcionalidades, integrando los cambios en la rama principal una vez que cada fase está completada y validada.

Este conjunto de requisitos asegura un entorno robusto y bien configurado para abordar los desafíos específicos del proyecto, optimizando tanto el desarrollo como la implementación de soluciones avanzadas en visión por computadora y sistemas robóticos.

3.1.2. Herramientas y Tecnologías Utilizadas

Para asegurar un entorno de desarrollo eficiente en este proyecto, se han integrado varias herramientas y tecnologías clave que facilitan cada etapa del proceso.

- ★ **Visual Studio Code[2]:** Incorpora extensiones fundamentales como el paquete de Python, que mejora el autocompletado y proporciona sugerencias inteligentes durante la codificación. La extensión Error Lens permite visualizar errores directamente dentro del código, lo cual agiliza la detección y corrección de problemas. Asimismo, Autodocstring genera documentación automáticamente basada en la estructura del código, asegurando una documentación detallada y coherente.
- ★ **Arquitectura del proyecto:** Se han desarrollado cuatro componentes principales correspondientes a cada una de las fases planificadas. Estos componentes se comunican entre sí mediante proxys y conexiones directas entre puertos, lo cual permite una implementación modular y coordinada de tareas específicas.
- ★ **Librerías:** Desempeñan roles críticos en diferentes aspectos del proyecto: NumPy se encarga de la preparación de fotogramas para su procesamiento, OpenCV proporciona la interfaz de usuario y facilita el procesamiento de imágenes y videos, PyTorch es utilizado para la creación, entrenamiento y predicción de redes neuronales, mientras que pyrealsense2 facilita la conexión con la cámara Intel RealSense para la captura y recepción de fotogramas en tiempo real o de un archivo previamente generado.
- ★ **GitHub:** Gestiona el control de versiones, siguiendo una estrategia de ramificación donde cada fase del proyecto se desarrolla en ramas separadas. Una vez que una fase está completa y probada, se fusiona con la rama principal. Esta metodología evita la proliferación de versiones y proporciona una estructura clara de qué partes están en desarrollo y cuáles están listas para implementarse.
- ★ **Documentación de librerías:** La mayoría de las librerías utilizadas está

3.1. FASE 1. PREPARACIÓN DEL ENTORNO

ampliamente disponible en línea, lo que facilita el acceso a guías de instalación, referencias de API y ejemplos prácticos. Sin embargo, la experiencia con pyrealsense2 ha destacado la importancia de recursos adicionales como la asistencia de ChatGPT[3] para superar desafíos específicos y avanzar en el desarrollo del proyecto.

Este conjunto de herramientas y tecnologías garantiza un entorno de desarrollo robusto y bien configurado, optimizando tanto la eficiencia operativa como la calidad de las soluciones implementadas en el proyecto.

3.1.3. Configuración del Control de Versiones

Se utiliza GitHub[4] que es una herramienta que se basa en tener un repositorio online para poder llevar el guardado, la gestión de versiones y la gestión del trabajo en grupo (Aunque en este caso haya sido individual).

Primero, se debe instalar el paquete y configurar el mismo con los siguientes comandos en la terminal.

```
$ sudo apt-get install git
$ git config --global user.name "<username>"
$ git config --global user.email "<email>"
```

Cuando se trabaja en un proyecto utilizando Git, es esencial comprender y dominar los comandos más relevantes de esta herramienta de control de versiones. Git proporciona una variedad de comandos que permiten a los desarrolladores gestionar eficientemente el historial de cambios, colaborar con otros miembros del equipo y mantener la integridad del proyecto. Desde la creación de ramas para el desarrollo de nuevas características hasta la fusión de cambios y la gestión de conflictos, cada comando desempeña un papel crucial en el flujo de trabajo del desarrollo de software.

- **Clonación del repositorio:** El comando de clonación es una herramienta fundamental que permite al usuario replicar un repositorio remoto en su entorno

local. Al proporcionar la “URL” del repositorio deseado, el comando descarga todos los archivos y la estructura de directorios asociada, brindando al usuario acceso completo al proyecto en su propia máquina. Este proceso es esencial para colaboradores y desarrolladores que desean contribuir al proyecto o trabajar en él de forma individual.

```
$ git clone <url>
```

- **Descarga de actualizaciones del repositorio:** Este comando descarga las nuevas actualizaciones del repositorio remoto y las guarda en el repositorio local del usuario. Es esencial hacerlo antes de modificar el repositorio local para evitar errores al subir las actualizaciones.

```
$ git pull
```

- **Subida de cambios al repositorio:** Permite subir los cambios realizados en el repositorio local al repositorio remoto. Los cambios se envían al servidor remoto, lo que actualiza el historial del repositorio remoto con las modificaciones realizadas. Además, es posible especificar la rama que se desea actualizar, lo que permite controlar qué cambios se envían a qué ramas en el repositorio remoto. Esto facilita la colaboración en equipo y la gestión de versiones del proyecto de manera eficiente.

```
$ git push origin <nombre_rama>
```

- **Gestión de versiones del repositorio:** Estos comandos permiten guardar versiones del proyecto asignadas a nombres descriptivos para facilitar la búsqueda y el entendimiento. Además, posibilitan volver a estas versiones anteriores cuando sea necesario, actuando como puntos de guardado del proyecto.

3.1. FASE 1. PREPARACIÓN DEL ENTORNO

```
$ git add .  
$ git commit -m "<Mensaje de version>"
```

- **Gestión de ramas del repositorio:** Los siguientes comandos posibilitan la creación de ramas para desarrollo, la actualización de ramas en el repositorio remoto, el borrado de ramas específicas y la fusión de ramas para guardar los cambios en la rama principal:

```
$ git branch                # Lista las ramas  
$ git branch <nombre_rama>  # Crea la rama  
$ git branch -[d/D] <nombre_rama> # Elimina la rama  
$ git checkout <nombre_rama>  # Cambia la rama  
$ git merge <nombre_rama>     # Combina dos ramas
```

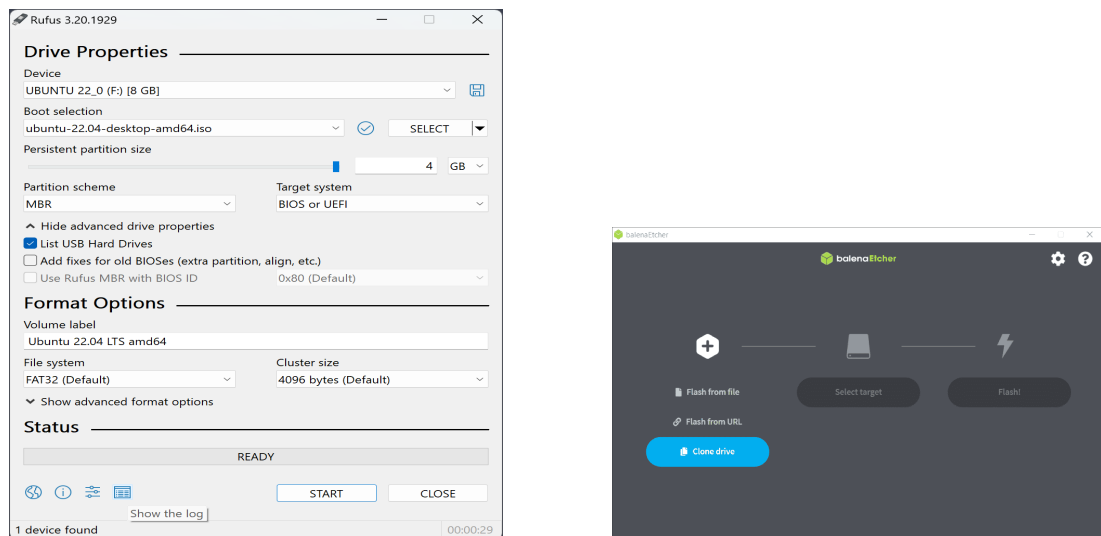
3.1.4. Instalación y Configuración

Ubuntu 22.04.4 LTS

La instalación de Ubuntu 22.04 LTS[5] es un proceso fundamental para configurar adecuadamente el entorno de desarrollo. A continuación se detalla el procedimiento desde la creación del USB booteable hasta la configuración post-instalación.

1. **Creación del USB Booteable:** Para crear un USB booteable con Ubuntu 22.04 LTS, se recomienda utilizar herramientas como Rufus[6] (en Windows), balenaEtcher[7] (multiplataforma) o la herramienta de creación de discos de Ubuntu si ya se está utilizando otro sistema Linux.

CAPÍTULO 3. SOLUCIÓN PROPUESTA



(a) Interfaz de Rufus.

(b) Interfaz de BalenaEtcher.

Figura 3.1: Generación USB Booteable.

El proceso de creación es el siguiente

- ★ Descarga la imagen ISO de Ubuntu 22.04[5] desde el sitio web oficial de Ubuntu.
- ★ Abre la herramienta seleccionada para la creación del USB booteable.
- ★ Selecciona la imagen ISO descargada y el dispositivo USB adecuado.
- ★ Inicia el proceso de creación y espera a que se complete. Esto puede llevar algunos minutos dependiendo de la velocidad del USB y del sistema.

2. **Gestión del Arranque desde USB:** Se debe reiniciar o encender el dispositivo y acceder a la configuración de la BIOS. Esto generalmente se hace presionando una tecla específica al iniciar (como F2, F12, Esc o Supr, dependiendo del fabricante).

Una vez dentro, se debe localizar la sección de configuración de arranque y ajustar la prioridad para que el dispositivo USB como se observa en la Figura 3.2 que contiene el nuevo sistema operativo creado en el paso anterior, sea el primero en la lista.

3.1. FASE 1. PREPARACIÓN DEL ENTORNO

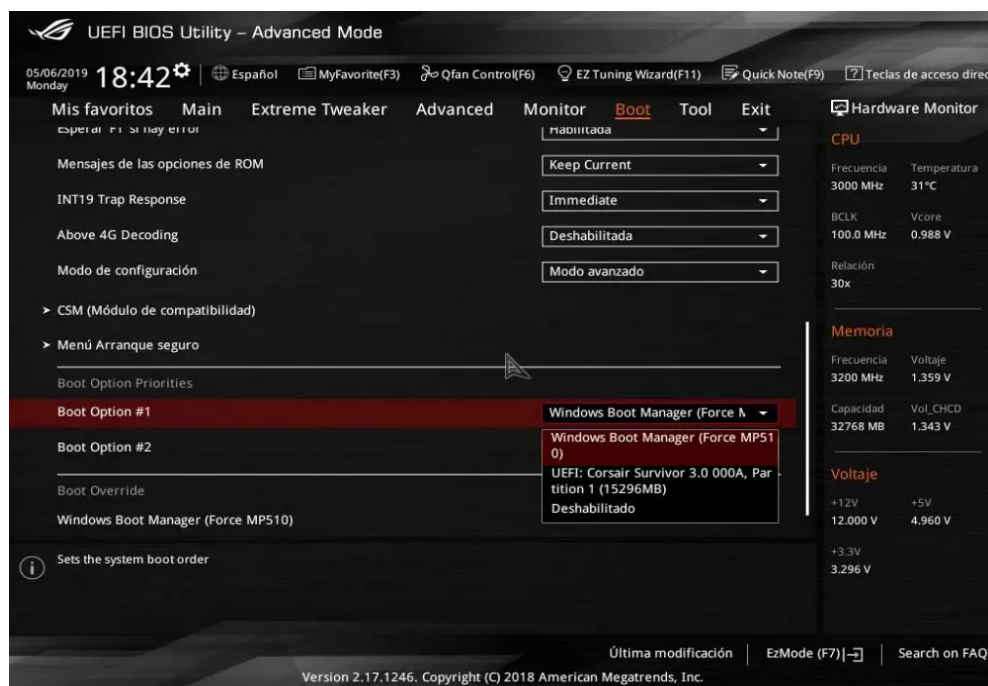


Figura 3.2: Selección de arranque de USB Booteable.

Tras esto, se guardan los cambios realizados en la BIOS y se reinicia el dispositivo.

Se selecciona el dispositivo USB que contiene el booteable de Ubuntu 22.04 LTS y se espera a que cargue el instalador de Ubuntu.

3. **Proceso de instalación de Ubuntu 22.04 LTS:** Una vez que se carga el instalador de Ubuntu, aparecerá una pantalla inicial donde puedes elegir la configuración del teclado y otros ajustes regionales. Selecciona “Instalar Ubuntu” para comenzar con el proceso de instalación.

Durante la instalación, se te guiará a través de la configuración de particiones del disco. Puedes elegir entre la instalación automática o personalizada de las particiones.

Define el espacio para la instalación como se muestra en la Figura 3.3 y elige el tipo de instalación según tus necesidades (por ejemplo, instalación junto a otro sistema operativo o limpieza completa del disco).

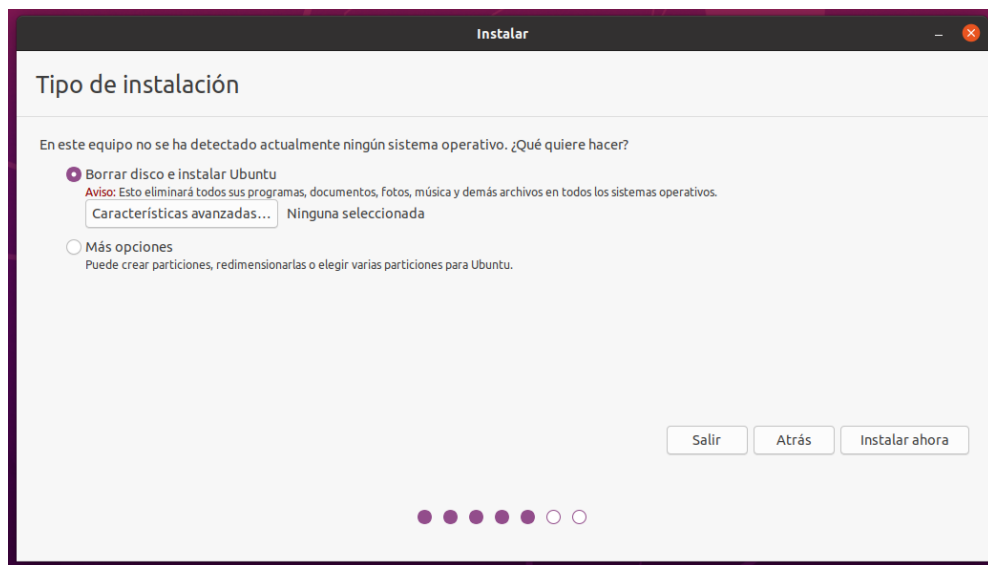


Figura 3.3: Menu de selección del tipo de instalación.

Completa la configuración inicial del sistema, que incluye la creación de cuentas de usuario, configuración de red y otras preferencias personales. Actualiza el sistema operativo y los paquetes instalados para asegurar que tienes la versión más reciente y los últimos parches de seguridad.

Siguiendo estos pasos detallados, podrás instalar correctamente Ubuntu 22.04 LTS en tu sistema, configurando adecuadamente el entorno de desarrollo para continuar con las siguientes fases del proyecto.

VS Code

Para configurar adecuadamente el entorno de desarrollo, es fundamental instalar Visual Studio Code[2] y las extensiones necesarias. A continuación se detallan los pasos para la instalación de VS Code y la configuración de sus extensiones principales. El proceso de instalación difiere levemente en función de si el sistema operativo es Windows o Ubuntu.

1. **Instalación de VS Code:** Se lleva a cabo la instalación de la última versión.

Posteriormente se inicia el proceso de instalación bien por medio del ejecutable (Windows) o bien añadiendo el paquete e instalándolo (Ubuntu)

3.1. FASE 1. PREPARACIÓN DEL ENTORNO

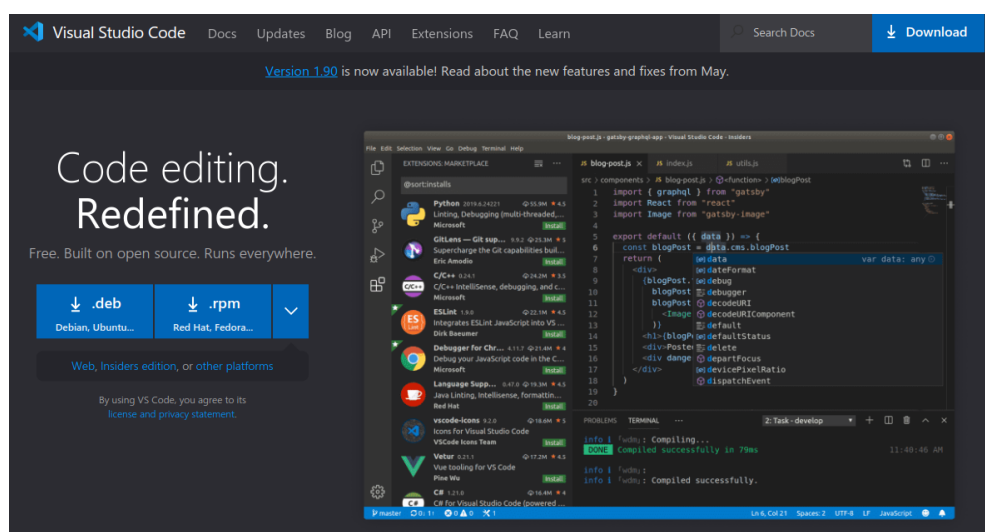


Figura 3.4: Pagina web de VS Code.

2. **Configuración de extensiones:** Al entorno de desarrollo se le pueden agregar un conjunto de extensiones que facilitan la creación y el proceso de proyectos. A continuación, se referenciarán las más importantes para el proyecto.

- ★ **Python:** Esta extensión facilita el autocompletado y la depuración de código en Python. Adicionalmente habilita al IDE a entender el código Python y su estructura.

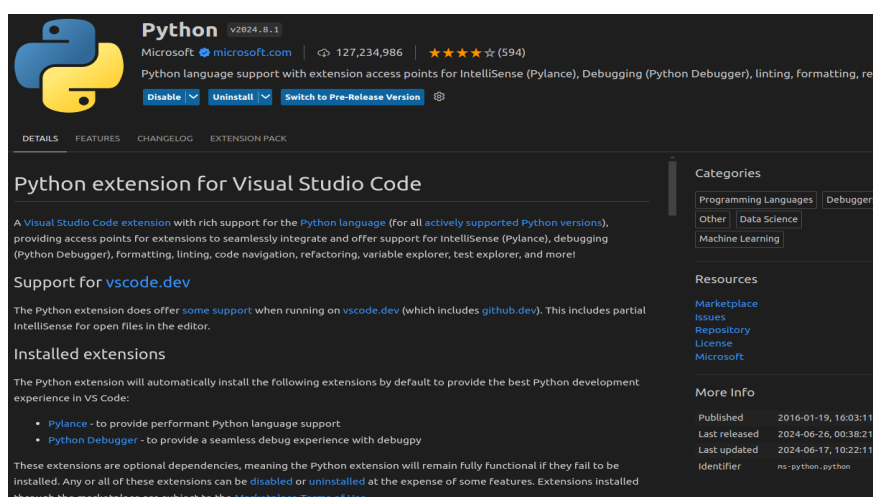


Figura 3.5: Extensión Python de VS Code.

- ★ **Error Lens:** Muestra los errores del código directamente en el editor, lo que ayuda a identificar y corregir errores más rápidamente.

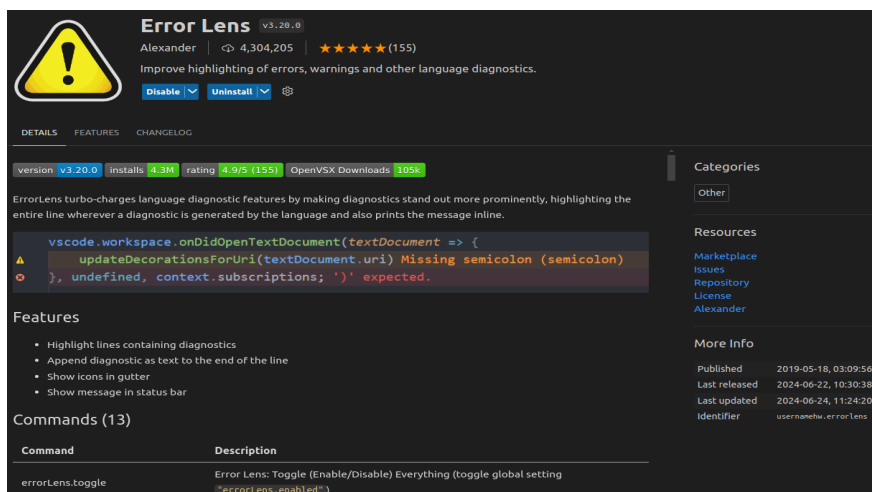


Figura 3.6: Extensión Error Lens de VS Code.

- ★ **Autodocstring:** Genera automáticamente la documentación para las funciones y clases de Python, facilitando la creación de docstrings conforme al formato estándar.



Figura 3.7: Extensión Autodocstring de VS Code.

3.1. FASE 1. PREPARACIÓN DEL ENTORNO

Librerías de Python

Para asegurar el correcto funcionamiento de las diversas fases del proyecto, es esencial instalar una serie de librerías de Python.

Se debe instalar el interprete de Python3 el cual trae adherido librerías básicas como algunas de las que se mencionan a continuación. Adicionalmente, es necesario un administrador de paquetes de Python para poder llevar a cabo la instalación de las librerías.

```
$ sudo apt-get install python3  
$ sudo apt-get install python3-pip
```

A continuación se mencionan las librerías básicas más recurrentes en el proyecto, junto con una breve descripción de cual es su funcionalidad dentro del código (De manera general).

- ★ **Librería Time:** Ofrece una variedad de funciones para la manipulación y el manejo del tiempo. Entre sus principales funcionalidades se incluyen la medición de intervalos de tiempo y la capacidad de retrasar la ejecución de programas.

Viene instalada por defecto.

- ★ **Librería NumPy:** Ofrece una amplia gama de funciones para la manipulación y el procesamiento de datos numéricos. Entre sus principales funcionalidades se incluyen operaciones matemáticas avanzadas, manejo de matrices y generación de números aleatorios.

Instalación de la librería por consola.

```
# Instalacion librería NumPy  
$ pip3 install numpy
```

- ★ **Librería OpenCV:** Proporciona una extensa colección de funciones para el

procesamiento y la manipulación de imágenes y videos. Entre sus principales funcionalidades se incluyen el reconocimiento de patrones, la detección de objetos, y la transformación y análisis de imágenes en tiempo real.

Instalación de la librería por consola.

```
# Instalacion librería OpenCV  
$ pip3 install opencv-python
```

- ★ **Librería OS:** Permite interactuar con el sistema operativo. Entre sus principales funcionalidades se incluyen la manipulación de archivos y directorios, la obtención de información del sistema, y la ejecución de comandos del sistema operativo.

Viene instalada por defecto.

- ★ **Librería Shutil:** Es una colección de funciones para la gestión de archivos y directorios a un nivel más avanzado. Entre sus principales funcionalidades se incluyen la eliminación recursiva de directorios y la gestión de permisos y propiedades de archivos.

Viene instalada por defecto.

- ★ **Librería SYS:** Permite interactuar con el intérprete de Python y el entorno de ejecución. Entre sus principales funcionalidades se incluyen la manipulación de la lista de argumentos de la línea de comandos y la gestión de la entrada y salida estándar.

Viene instalada por defecto.

- ★ **Librería JSON:** Ayuda a administrar datos en formato JSON (JavaScript Object Notation). Entre sus principales funcionalidades se incluyen la conversión de datos entre objetos de Python y cadenas JSON, así como la lectura y escritura de datos JSON desde y hacia archivos.

Viene instalada por defecto.

3.1. FASE 1. PREPARACIÓN DEL ENTORNO

- ★ **Librería Random:** Permite la generación de números aleatorios y realizar operaciones relacionadas con la aleatoriedad. Entre sus principales funcionalidades se incluyen la selección aleatoria de elementos de una secuencia y la mezcla aleatoria de secuencias.

Viene instalada por defecto.

RoboComp

RoboComp[8] es un framework open-source que proporciona una infraestructura para el desarrollo de aplicaciones robóticas. Su objetivo es simplificar la programación de robots mediante el uso de componentes independientes que pueden ser desarrollados, testeados y desplegados de manera modular. Esto permite a los desarrolladores centrarse en la funcionalidad específica de cada componente sin preocuparse por la integración global del sistema.

El proceso de instalación y configuración puede ser complejo, especialmente si no se siguen correctamente las instrucciones o si surgen problemas no documentados. A continuación, se describe el proceso que siguió el autor del Trabajo de Fin de Grado (TFG) para la instalación y configuración de RoboComp.

1. **Instalación de dependencias:** Para asegurar un desarrollo fluido y evitar errores durante la ejecución de los componentes del proyecto, es fundamental realizar la instalación de todas las librerías y paquetes necesarios de manera ordenada y cronológica. A continuación, se proporciona una guía clara para la instalación:

```
$ sudo apt update

$ sudo apt install python3 python3-pip cmake vim git wget \
libopenscenegraph-dev libgsl-dev

$ pip3 install vcstool

$ sudo apt install qt6-base-dev qt6-declarative-dev \
qt6-scxml-dev libqt6statemachineqml6 libqt6statemachine6 \
libbz2-dev libssl-dev zeroc-icebox zeroc-ice-all-dev \
libzeroc-icestorm3.7 libeigen3-dev meld

$ pip install PySide6 zeroc-ice

$ mkdir ~/software 2> /dev/null; cd ~/software && \
git clone https://github.com/GillesDebunne/libQGLViewer.git

$ cd libQGLViewer && qmake6 *.pro && make -j12 && \
sudo make install && sudo ldconfig
```

2. **Descarga de .repo:** En la instalación de Robocomp se usa la herramienta *vcstool*, la cual es un gestor de repositorios, y permite la descarga de todos los repositorios necesarios que estén definidos en el fichero *.repo* con un solo comando.

```
$ cd && wget https://raw.githubusercontent.com/robocomp/\
robocomp2/main/robocomp.repos
```

3. **Modificación del PATH:** Robocomp instalará varias herramientas, para que se puedan usar es imprescindible añadir unas rutas al PATH, este para que sea

3.1. FASE 1. PREPARACIÓN DEL ENTORNO

persistente se modificará el archivo `/.bashrc` mediante los comandos:

```
$ echo "export ROBCOMP=/home/robocomp/robocomp" \  
>> ~/.bashrc  
$ echo "export PATH=$PATH:/home/$USER/.local/bin"\  
>> ~/.bashrc  
$source ~/.bashrc
```

De esta forma el archivo `/.bashrc` quedara como se muestra en la Figura 3.8

```
119 # Importaciones para el proyecto  
120 export ROBCOMP=/home/robocomp/robocomp  
121 export PATH=$PATH:/home/robocomp/.local/bin
```

Figura 3.8: Modificación del PATH para añadir las herramientas.

4. **Descarga de paquetes de Robocomp:** Se descargarán todos los repositorios necesarios con herramienta *vcstool*, crearán enlaces simbólicos y construirá la carpeta de componentes, de la siguiente manera:

```
$ vcs import . < robocomp.repos --recursive  
  
$ sudo ln -s /home/$USER /home/robocomp  
  
$ cd /home/robocomp/robocomp  
$ ln -s core/cmake cmake  
$ ln -s core/classes classes  
  
mkdir components
```

5. **Instalación de Robocomp mediante pip:** Siendo este de los últimos pasos de la instalación, se instalará *robocompcli* mediante la herramienta *pip*. La herramienta *robocompcli* permitirá la generación de componentes Robocomp e instalación de

otras librerías como *rcnode*, que permite la creación de un servidor fundamental para activar la base del robot y habilitar el joystick utilizado para su control.

```
$ pushd . && cd tools/cli/ && pip install . && popd
```

6. **Simplificación del comando *rcnode*:** Se creará un alias para el uso rápido y sencillo de *rcnode*, para ello se modificara el archivo */.bashrc* mediante los comandos:

```
$ echo "alias rcnode='bash /home/robocomp/robocomp/tools/\nrcnode/rcnode.sh&'" >> ~/.bashrc\n$ source ~/.bashrc
```

Yakuake

Se trata de un emulador de terminal desplegable para entornos de escritorio KDE en sistemas operativos basados en Linux. Su función principal es proporcionar una terminal rápida y accesible que se despliega desde la parte superior de la pantalla (por defecto) con una combinación de teclas o mediante un clic en un icono en la bandeja del sistema.

Para instalar Yakuake[9], ejecuta el siguiente comando en la terminal:

```
$ sudo apt-get install yakuake
```

Al seguir estos pasos, se asegura que el entorno de desarrollo esté correctamente configurado, minimizando la posibilidad de errores durante la fase de implementación y ejecución del proyecto.

3.2. Fase 2. Generación de archivo de vídeo RGBD

3.2.1. Introducción

En el contexto de la robótica asistida por inteligencia artificial, la generación de un archivo de video RGBD juega un papel fundamental en la creación de datasets robustos y de alta calidad. Este archivo de video no solo proporciona los datos necesarios para el entrenamiento de algoritmos de detección y seguimiento de personas, sino que también sirve como base para el desarrollo de sistemas autónomos capaces de interactuar de manera segura y eficiente con su entorno.

Esta fase se enfoca en la implementación y desarrollo de un código que pueda llevar a cabo la obtención de este archivo de video RGBD. Primero, se establece la infraestructura necesaria para la grabación de fotogramas utilizando una cámara Intel RealSense. Se detalla la configuración inicial del entorno, incluyendo la gestión de la interfaz de usuario para la visualización en tiempo real de lo que se está grabando. Asimismo, se aborda la implementación de controles de error robustos para garantizar la integridad del proceso de grabación.

Después en la sección 3.2.5, se profundiza en el código y la estructura del proyecto, destacando el rol central de la clase `SpecificWorker` en la coordinación y ejecución de las tareas críticas. Se explora cómo se gestionan los fotogramas capturados utilizando bibliotecas como `pyrealsense2` y `opencv`, junto con el manejo de errores mediante la detección y corrección de posibles problemas durante la ejecución del programa.

Además de describir la metodología y la implementación técnica, se presentan resultados específicos obtenidos durante las sesiones de grabación. Estos resultados incluyen métricas de calidad del video, como la cantidad de fotogramas capturados y posibles desafíos técnicos enfrentados durante el proceso.

En resumen, este estudio proporciona una visión detallada del proceso completo para la generación de archivos de video RGBD, resaltando la importancia de cada fase en la obtención de datos confiables para el desarrollo de sistemas inteligentes en el ámbito robótico. Los detalles presentados aquí servirán como referencia fundamental

para futuras investigaciones y mejoras en la eficiencia y precisión de los sistemas de visión y seguimiento implementados.

3.2.2. Objetivos

Esta fase del proyecto tiene como propósito principal la generación de un archivo de video RGBD que cumpla con estándares de calidad y confiabilidad adecuados para la posterior creación de un dataset utilizado en la investigación y desarrollo de sistemas robóticos asistidos por inteligencia artificial. Los objetivos específicos abordados durante esta fase incluyen:

- ★ **Implementación de la interfaz de usuario:** Desarrollar una interfaz de usuario intuitiva que permita la visualización en tiempo real de los fotogramas capturados por la cámara RealSense. Esta interfaz debe ser capaz de pausar, reanudar o detener la grabación según las necesidades del usuario, proporcionando así un control efectivo sobre el proceso de captura.
- ★ **Control de errores:** Implementar mecanismos robustos para detectar y manejar posibles errores durante la conexión con la cámara, la inicialización de la grabación y el almacenamiento de los fotogramas. Esto incluye la verificación de la correcta configuración de la cámara, la detección de archivos duplicados en la ruta de destino y la gestión de excepciones para garantizar la continuidad operativa sin interrupciones significativas.
- ★ **Protección de datos:** Establecer medidas de protección para evitar la sobreescritura accidental de grabaciones anteriores. Esto implica verificar la existencia previa de archivos en la ruta especificada y notificar al usuario para evitar pérdidas de datos inadvertidas.
- ★ **Generación de dataset consistente:** Asegurar que el archivo de video resultante contenga un número suficiente de fotogramas para formar un dataset coherente y útil. Se establece como referencia un mínimo de 30 fotogramas por segundo

3.2. FASE 2. GENERACIÓN DE ARCHIVO DE VÍDEO RGBD

de grabación, con el objetivo de capturar una cantidad significativa de datos que permitan el entrenamiento efectivo de modelos de inteligencia artificial.

- ★ **Verificación de calidad y funcionalidad:** Evaluar la correcta implementación del sistema mediante la verificación del número de fotogramas capturados y la ausencia de errores críticos durante el proceso. Esto incluye pruebas funcionales para confirmar que la cámara se conecta correctamente, la captura de fotogramas es estable y la interfaz de usuario responde de manera adecuada a las interacciones del usuario.
- ★ **Preparación para fases subsecuentes:** Establecer una base sólida para futuras mejoras y expansiones del sistema, considerando aspectos como la optimización de la velocidad de captura, la exploración de escenarios de uso más diversos y la implementación de funcionalidades adicionales para aumentar la versatilidad y eficiencia del sistema.

Estos objetivos proporcionan un marco estructurado para la fase de generación de archivo de video RGBD, delineando claramente las metas a alcanzar para asegurar el éxito del proyecto y la calidad del dataset resultante.

3.2.3. Explicaciones técnicas

Durante la implementación de esta fase del proyecto, se utilizaron diversas técnicas y herramientas tecnológicas clave para garantizar el funcionamiento efectivo y confiable del sistema de generación de archivo de video RGBD. A continuación, se detallan las principales explicaciones técnicas involucradas:

- **Interfaz de usuario (UI):** Se desarrolló una interfaz de usuario utilizando la biblioteca OpenCV en Python. Esta interfaz permite la visualización en tiempo real de los fotogramas capturados por la cámara RealSense, facilitando al usuario la monitorización del proceso de grabación. Mediante esta interfaz, el usuario podría pausar, reanudar o detener la grabación con simples interacciones de

teclado, proporcionando un control interactivo sobre el sistema. La interfaz de usuario es crucial para ofrecer retroalimentación inmediata al usuario, mejorando la usabilidad y permitiendo ajustes en tiempo real basados en las condiciones de grabación.

- **Recepción de fotogramas no bloqueante:** El método compute se ejecuta periódicamente cada 33 ms y utiliza llamadas no bloqueantes para la recepción de fotogramas desde la cámara RealSense. Una llamada no bloqueante permite al programa continuar su ejecución incluso si no se reciben fotogramas de inmediato, en lugar de esperar indefinidamente (como en una llamada bloqueante). Esta técnica asegura que el sistema siga funcionando de manera fluida, solicitando el siguiente fotograma si uno falla, lo que mejora la eficiencia y reduce el riesgo de bloqueos que podrían interrumpir la grabación.
- **Protección de datos previos:** Para evitar la sobreescritura accidental de grabaciones anteriores, se implementaron mecanismos de protección de datos mediante el uso de flags. Antes de iniciar una nueva grabación, el sistema verifica si ya existe un archivo con el mismo nombre en la ruta especificada. Si encuentra un archivo existente, se levanta un flag que alerta al usuario y evita la sobreescritura. Este sistema de flags asegura que los datos previos se mantengan intactos, proporcionando una capa adicional de seguridad y confiabilidad en la gestión de archivos.
- **Control de errores:** La robustez del sistema se garantizó mediante la implementación de un exhaustivo control de errores. Se utilizaron estructuras de manejo de excepciones en Python para capturar y gestionar errores de manera eficiente. Entre los errores controlados se incluyen problemas de conexión con la cámara, uso concurrente de la cámara en otra instancia, y la existencia previa de archivos de destino. Además, se incorpora la sentencia `sys.exit("mensaje")` para detener la ejecución del código y mostrar mensajes de error detallados, facilitando la identificación y resolución de problemas. Este manejo proactivo

3.2. FASE 2. GENERACIÓN DE ARCHIVO DE VÍDEO RGBD

de errores asegura que el sistema pueda responder adecuadamente a diversas situaciones imprevistas, manteniendo su estabilidad y funcionalidad.

Estas explicaciones técnicas proporcionan una visión detallada de los componentes y procesos fundamentales implementados durante la fase de generación de archivo de video RGBD, destacando cómo cada tecnología y metodología contribuyó al éxito y funcionamiento efectivo del sistema.

3.2.4. Librerías principales

Para el desarrollo de este proyecto, se han utilizado diversas librerías que han jugado roles fundamentales en la implementación de las funcionalidades necesarias para la generación de archivos de video RGBD. Entre ellas, la más destacada es `pyrealsense2`, acompañada por otras librerías esenciales como `OpenCV`, `NumPy` y `JSON`.

- **`pyrealsense2`[10]**: Es la librería principal y es una herramienta indispensable para la captura y procesamiento de datos de cámaras Intel RealSense. Esta librería proporciona una interfaz robusta y eficiente para interactuar con los dispositivos RealSense, permitiendo la configuración de parámetros de captura, la sincronización de flujos de datos y la grabación de videos en formato RGBD. En este proyecto, `pyrealsense2` se utiliza para establecer la conexión con la cámara, configurar las propiedades de los flujos de color y profundidad, y capturar los fotogramas que se almacenan posteriormente en el disco. Además, facilita el manejo de dispositivos RealSense a través de métodos que permiten la activación y desactivación de la grabación, así como la gestión de los archivos de destino.
- **`OpenCV`[11]**: Es conocida por sus capacidades en procesamiento de imágenes y visión por computadora, `OpenCV` permite la manipulación y visualización de los fotogramas capturados. En este proyecto, se utiliza principalmente para mostrar los fotogramas en la interfaz de usuario, permitiendo al usuario visualizar en

tiempo real lo que está siendo grabado. Además, OpenCV es crucial para guardar los fotogramas en el disco en formatos de imagen comunes como PNG o JPG, utilizando funciones como `cv2.imwrite`. Esta librería también facilita la captura de eventos del teclado, permitiendo al usuario pausar, reanudar o detener la grabación de manera interactiva.

- **NumPy[12]**: es esencial para el manejo de datos en forma de matrices y arrays, proporcionando una estructura de datos eficiente y herramientas matemáticas avanzadas. En este proyecto, NumPy se utiliza para convertir los datos de los fotogramas capturados por la cámara en arrays manejables, permitiendo su procesamiento y almacenamiento. La capacidad de NumPy para realizar operaciones matemáticas y de manipulación de datos de manera eficiente es crucial para manejar los grandes volúmenes de datos generados durante la grabación de videos RGBD.
- **JSON[13]** es una librería que se utiliza para la gestión de datos en formato JSON, facilitando el almacenamiento y la transferencia de datos estructurados. En el contexto de este proyecto, JSON es útil para guardar configuraciones y metadatos asociados con las grabaciones, como las propiedades de los flujos de datos y los parámetros de la cámara. Utilizar JSON permite que estos datos sean fácilmente accesibles y modificables, proporcionando una forma sencilla y eficiente de manejar la configuración y los metadatos del sistema.

En conjunto, estas librerías forman el núcleo tecnológico del proyecto, cada una contribuyendo con funcionalidades específicas que, en conjunto, permiten la captura, procesamiento, visualización y almacenamiento de videos RGBD de manera eficiente y efectiva. La combinación de `pyrealsense2`, OpenCV, NumPy y JSON proporciona una base sólida y versátil para el desarrollo de aplicaciones que requieren la manipulación avanzada de datos de cámaras, asegurando la fiabilidad y robustez del sistema implementado.

3.2.5. Desarrollo del código

En este apartado, se aborda los métodos clave de la clase encargada de la conexión con la cámara RGBD y el proceso de grabación en disco. Se explica cómo se establece la comunicación con el dispositivo, cómo se configuran los parámetros necesarios para la captura y el manejo eficiente del almacenamiento de la grabación. Para una visión detallada del código, se puede consultar la implementación de la clase en el repositorio de GitHub[14].

1. **compute() [65-80]:** El método compute es responsable de manejar la captura y procesamiento de fotogramas en tiempo real desde la cámara conectada. Primero, solicita a la cámara que envíe nuevos datos de fotogramas. Este proceso no bloquea la ejecución del programa, lo que significa que el programa continúa corriendo mientras espera los datos. La solicitud devuelve dos elementos: uno que indica si hay fotogramas disponibles y otro que contiene el conjunto de fotogramas capturados.

Si los fotogramas están disponibles, el método procede a mostrarlos al usuario utilizando la interfaz gráfica de OpenCV. Extrae los fotogramas de color y profundidad, los convierte en arrays y los presenta en ventanas para su visualización en tiempo real. Luego, incrementa un contador que lleva el registro del número de fotogramas procesados hasta el momento. Si no se reciben fotogramas, se detecta un error, y el programa se detiene, ya que la recepción continua de fotogramas es crucial para el funcionamiento del sistema.

2. **comprobacion_requisitos_minimos() [84-102]:** se encarga de verificar que se cumplan ciertas condiciones previas necesarias para el funcionamiento correcto del programa. Su propósito principal es asegurar que el entorno y los archivos necesarios estén en condiciones adecuadas antes de iniciar el procesamiento.

Primero, el método comprueba si ya existe un archivo de grabación en la ruta especificada y si el reemplazo de archivos no está habilitado. Si el archivo existe y el reemplazo no está permitido, el método detiene la ejecución del programa y

muestra un mensaje de error, indicando que la grabación no puede proceder.

Luego, verifica si el directorio donde se guardará el archivo de grabación realmente existe. Si el directorio no existe, el método detiene la ejecución del programa y muestra un mensaje de error, informando al usuario sobre la ausencia del directorio.

Finalmente, el método asegura que el archivo de grabación tenga la extensión correcta. Se separa el nombre del archivo y su extensión, y se verifica que la extensión sea .bag. Si la extensión no es la esperada, se muestra un mensaje de error y se detiene el programa.

3. **iniciar_conexion_camara_y_grabacion()** [104-140]: El método se encarga de establecer una conexión con la cámara y configurar el flujo de datos para la grabación. Primero, inicializa los objetos necesarios de la librería para manejar la cámara. Luego, crea un objeto de configuración para definir cómo se deben capturar y registrar los datos.

El método configura los flujos de datos de la cámara, especificando qué tipos de datos se van a recibir: fotogramas de color y fotogramas de profundidad. Para cada tipo de flujo, establece la resolución deseada y la tasa de fotogramas por segundo. También indica que se desea guardar el contenido en un archivo específico, configurando la grabación en la ruta establecida.

Una vez que se han configurado los flujos y la grabación, el método inicia la conexión con la cámara, lo que activa el flujo de datos. Finalmente, actualiza los valores de las variables de estado para reflejar que la conexión se ha establecido correctamente y registra el tiempo de inicio de la conexión. Además, informa al usuario que la configuración y conexión se han completado con éxito.

4. **interfaz_usuario()** [144-160]: Se encarga de mostrar al usuario los fotogramas capturados por la cámara en tiempo real. Primero, extrae los fotogramas de color y profundidad del conjunto de datos proporcionado por la cámara. Estos fotogramas se obtienen en un formato que se convierte en arrays de numpy para

3.2. FASE 2. GENERACIÓN DE ARCHIVO DE VÍDEO RGBD

facilitar su manipulación.

Luego, el método utiliza OpenCV para mostrar estos fotogramas en ventanas gráficas. Los fotogramas de color y profundidad se muestran en ventanas separadas con los nombres “Fotogramas Color” y “Fotogramas Profundidad”, respectivamente.

Después de mostrar los fotogramas, se utiliza un método de control de teclas para gestionar cualquier entrada del teclado del usuario, permitiendo una interacción básica, como cerrar el programa cuando se presiona una tecla específica. Se asegura de que la visualización sea fluida al asignar un tiempo de espera mínimo de 1 milisegundo entre actualizaciones de las ventanas de visualización.

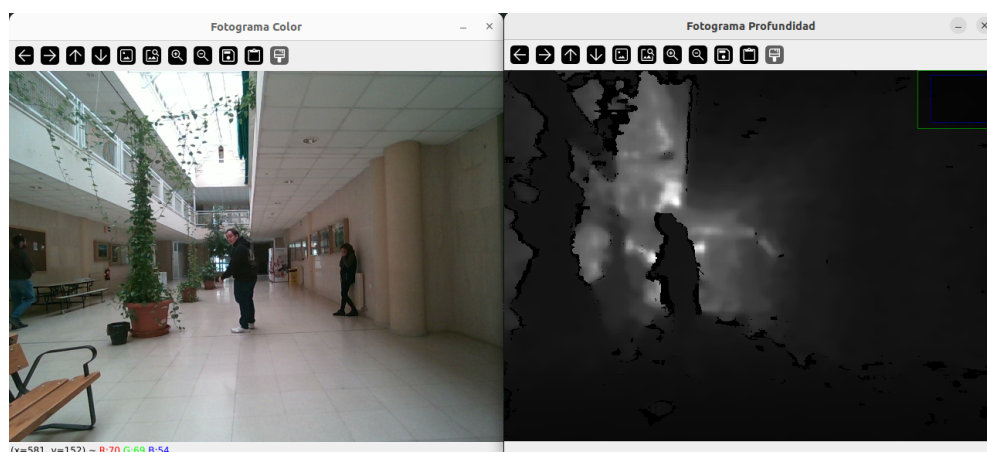


Figura 3.9: Interfaz de usuario fase 2.

5. **generacion_de_resultados()** [181-202]: Se encarga de calcular y guardar los resultados del proceso de grabación al finalizar la ejecución del programa. Primero, calcula el tiempo total de conexión restando el tiempo actual del tiempo de inicio registrado. Luego, crea un diccionario que contiene la información relevante, como el tiempo de conexión, el número de fotogramas guardados, y la ruta del archivo de grabación.

El método muestra esta información en la consola para proporcionar un resumen rápido al usuario. También guarda estos resultados en un archivo JSON en la ruta especificada. Antes de escribir el archivo, verifica que el directorio donde se

guardará el archivo JSON existe. Si el directorio no está presente, la operación de guardado no se realiza.

Este desarrollo de código abarca la estructura y funcionalidad del programa, desde la inicialización de la conexión con la cámara hasta la gestión de los fotogramas y la generación de resultados, proporcionando un flujo de trabajo eficiente y robusto para la captura y análisis de video con cámaras Intel RealSense.

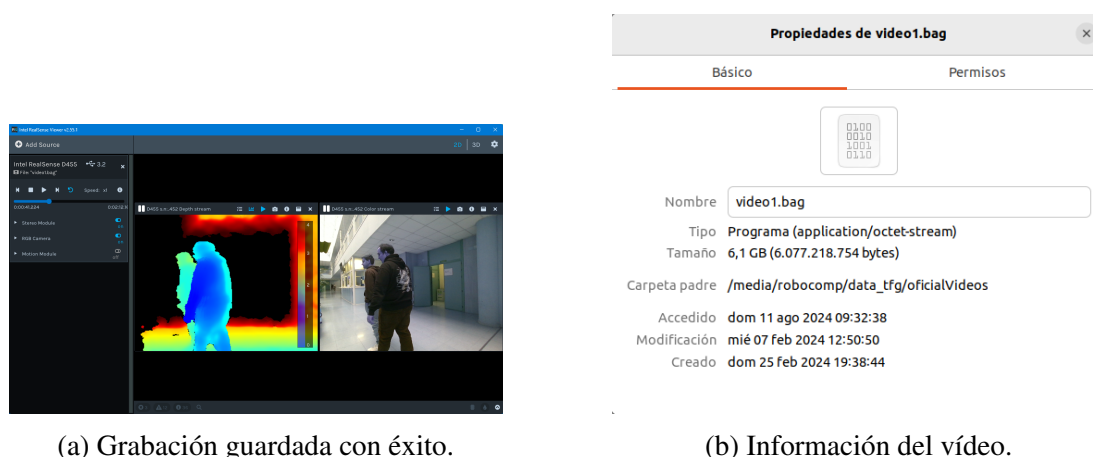
3.2.6. Resultados

Durante esta fase del proyecto, se desarrolla un código que automatiza el proceso de grabación de vídeo utilizando la librería pyrealsense2. Este código genera dos archivos clave como resultado de su ejecución.

El primero es un archivo de grabación con extensión “.bag”, que se almacena automáticamente en el directorio designado por el usuario. Este archivo contiene la grabación completa del vídeo capturado, el cual incluye dos flujos de datos: el de color (RGB) y el de profundidad (D). El vídeo se graba a una resolución de 640 x 480 píxeles, con una tasa de 30 fotogramas por segundo (FPS).

Cada fotograma ocupa un tamaño aproximado de 1.46 MB, lo que permite calcular fácilmente el tamaño total del archivo en función de la duración del vídeo. Por ejemplo, con la configuración actual, se puede estimar que el tamaño total del archivo por minuto de grabación es de aproximadamente 2.57 GB, lo que brinda una referencia clara para la gestión del almacenamiento necesario para cada sesión de grabación.

3.2. FASE 2. GENERACIÓN DE ARCHIVO DE VÍDEO RGBD



(a) Grabación guardada con éxito.

(b) Información del vídeo.

Figura 3.10: Información del archivo de grabación.

El segundo archivo generado es un archivo JSON que registra información detallada sobre el vídeo almacenado. Entre los datos incluidos se encuentran la duración total del proceso de grabación, el número total de fotogramas capturados y la ruta exacta donde se ha guardado el archivo de vídeo .bag (Como se muestra en la figura 3.10b). Estos resultados son fundamentales para el análisis y la gestión de los datos capturados, proporcionando una visión clara y estructurada de las características del vídeo almacenado.

```
1{"Tiempo de conexion" : 132.016 segundos,
2 "Numero de frames guardados" : 6907 fotogramas,
3 "Ruta destino de la grabacion" : /media/robocomp/data_tfg/oficialVideos/video1.bag
```

Figura 3.11: Ejemplo del archivo JSON generado.

3.2.7. Conclusiones

El desarrollo del código para la captura y procesamiento de datos de una cámara Intel RealSense utilizando las bibliotecas PySide2, pyrealsense2, OpenCV, y NumPy ha demostrado ser una solución eficiente y robusta para manejar grandes volúmenes de datos de sensores. El uso de estos componentes permitió la implementación de una aplicación que no solo captura y visualiza los datos en tiempo real, sino que también almacena los resultados de manera estructurada y legible.

El archivo de video en formato .bag es una elección acertada debido a su capacidad

para manejar datos sincronizados de múltiples flujos de sensores, lo que es crucial para aplicaciones que requieren precisión y eficiencia en el manejo de datos complejos. Este formato garantiza que los datos de color y profundidad capturados por la cámara se almacenen de manera eficiente y puedan ser reproducidos y analizados posteriormente con las herramientas adecuadas.

El archivo JSON generado proporciona una forma clara y estructurada de almacenar y revisar los resultados de la grabación. Su legibilidad y facilidad de uso permiten una rápida interpretación y análisis, mientras que su amplia compatibilidad asegura que los datos puedan ser integrados y utilizados en diferentes plataformas y aplicaciones.

La verificación de la integridad de los resultados a través de la comparación entre los fotogramas esperados y los fotogramas reales capturados es un paso crítico que garantiza la fiabilidad del sistema. Este proceso de validación es esencial para asegurar que la captura de datos sea precisa y que cualquier pérdida de datos sea identificada y cuantificada.

En resumen, el proyecto ha logrado crear una herramienta eficaz para la captura y almacenamiento de datos de cámaras Intel RealSense, con un enfoque en la precisión, eficiencia y facilidad de análisis.

3.2.8. Ampliaciones a futuro

El código desarrollado ofrece una base sólida para la captura y procesamiento de datos de cámaras Intel RealSense, pero siempre hay oportunidades para mejorar y ampliar sus capacidades. A continuación, se presentan algunas sugerencias conceptuales para futuras ampliaciones que podrían optimizar su rendimiento y funcionalidad.

- **Segmentación de archivos de vídeo:** Actualmente, el sistema graba todo el contenido en un único archivo .bag. Una mejora significativa sería implementar una funcionalidad que divida el video en múltiples archivos más pequeños, por ejemplo, cada 2 GB. Esto podría facilitar la gestión y el análisis de los datos, así

3.2. FASE 2. GENERACIÓN DE ARCHIVO DE VÍDEO RGBD

como mejorar la fiabilidad del almacenamiento, ya que se reduciría el riesgo de corrupción de archivos grandes.

- **Mejora en el control de errores:** El control de errores puede ser ampliado para manejar una mayor variedad de posibles fallos y condiciones excepcionales. Por ejemplo:

- ★ **Detección y manejo de pérdida de fotogramas:** Implementar mecanismos para detectar y reportar la pérdida de fotogramas en tiempo real, permitiendo acciones correctivas inmediatas.

- ★ **Alertas y notificaciones:** Configurar el sistema para que envíe alertas o notificaciones en caso de errores críticos, como la desconexión de la cámara o problemas de almacenamiento.

- ★ **Registro de errores:** Crear un sistema de logging más detallado que registre todos los errores y eventos significativos, facilitando la depuración y el análisis post-mortem.

- **Mejora de la interfaz de usuario:** La interfaz de usuario puede mejorarse para proporcionar una experiencia más interactiva y controlada. Algunas sugerencias incluyen:

- ★ **Pausar y reanudar grabación:** Permitir que el usuario pause y reanude la grabación mediante teclas específicas, ofreciendo mayor flexibilidad en el control del proceso de captura.

- ★ **Visualización en tiempo real:** Añadir gráficos o estadísticas en tiempo real que muestren información relevante, como el número de fotogramas capturados, el tiempo de conexión y el tamaño del archivo generado.

- ★ **Menús interactivos:** Incluir menús o diálogos interactivos que permitan al usuario ajustar parámetros de la grabación, como la resolución o la tasa de fotogramas, sin necesidad de detener la grabación.

- **Optimización del rendimiento:** La eficiencia del sistema puede ser mejorada mediante varias optimizaciones:
 - ★ **Paralelización de procesos:** Implementar técnicas de paralelización para dividir la carga de trabajo entre múltiples hilos o procesos, mejorando el rendimiento y reduciendo los tiempos de latencia.
 - ★ **Compresión de datos:** Aplicar métodos de compresión en tiempo real para reducir el tamaño de los archivos de video sin comprometer la calidad de los datos, facilitando el almacenamiento y la transferencia.
- **Almacenamiento y procesamiento en la nube:** Para mejorar la accesibilidad y escalabilidad, se podría integrar el sistema con servicios de almacenamiento y procesamiento en la nube:
 - ★ **Almacenamiento en la nube:** Subir automáticamente los archivos de video y los archivos JSON a un servicio de almacenamiento en la nube, garantizando la seguridad y accesibilidad de los datos.
 - ★ **Procesamiento distribuido:** Utilizar servicios de procesamiento en la nube para analizar grandes volúmenes de datos, permitiendo un análisis más rápido y eficiente.

3.3. Fase 3. Generación y clasificación de dataset

3.3.1. Introducción

El objetivo principal de esta fase del proyecto es la generación de un dataset clasificado a partir de fotogramas consecutivos de un vídeo, enfocado en la identificación y clasificación de personas como “persona objetivo” y “personas no objetivo”. Esta fase se divide en dos etapas: primero, la extracción y almacenamiento de fotogramas del vídeo; segundo, la generación del dataset a partir de esos fotogramas almacenados.

3.3. FASE 3. GENERACIÓN Y CLASIFICACIÓN DE DATASET

Se utiliza Python como lenguaje de programación debido a su eficacia en el procesamiento de imágenes y aprendizaje automático. Las librerías esenciales incluyen OpenCV para la manipulación de imágenes y vídeos y NumPy para operaciones numéricas. La detección de personas en las imágenes se realiza mediante la red neuronal YOLO[15] (You Only Look Once).

La precisión y calidad del dataset se aseguran mediante la asignación inicial de la persona objetivo en los primeros fotogramas del vídeo. Posteriormente, se sigue una ruta comparando la escala de grises de la región de interés (ROI) con la persona objetivo del último fotograma donde se le vio. Solo se aceptan detecciones con una precisión predeterminada o más, utilizando técnicas como el Error Cuadrático Medio (MSE) para comparar la escala de grises de imágenes de igual tamaño.

Para la generación de un dataset clasificado a partir de fotogramas consecutivos de un vídeo, se recomienda utilizar una Unidad de Computación Básica (NUC). Aunque no se requiere un equipo extremadamente potente, es esencial que el NUC cumpla con ciertos estándares mínimos para asegurar un rendimiento adecuado. A continuación, se detallan las características mínimas recomendadas:

- **CPU:** Un procesador de 4 núcleos proporciona suficiente capacidad de procesamiento para manejar las tareas de extracción y almacenamiento de fotogramas, así como el procesamiento necesario para la detección de personas utilizando la red YOLO.
- **GPU:** Una GPU con compatibilidad CUDA es esencial para acelerar el procesamiento de imágenes y el entrenamiento de modelos de redes neuronales significativamente, lo cual es crítico para mantener la eficiencia del proyecto (Aunque no es imprescindible).
- **RAM:** Una cantidad adecuada de memoria RAM es necesaria para manejar grandes volúmenes de datos y para la ejecución eficiente de los algoritmos de procesamiento de imágenes y aprendizaje automático.
- **Almacenamiento:** Un almacenamiento amplio es necesario para guardar los

vídeos y el dataset generado, garantizando así que se puedan almacenar todos los datos sin problemas de espacio (También se pueden usar discos externos).

Los NUCs son dispositivos compactos y eficientes que ofrecen una buena combinación de rendimiento y tamaño. Son ideales para tareas intensivas de procesamiento de vídeo y datos debido a su diseño compacto y capacidad de personalización para cumplir con los requisitos mínimos de hardware necesarios para este proyecto. Además, su tamaño reducido facilita su integración en entornos de trabajo diversos sin ocupar mucho espacio.

3.3.2. Objetivos

El proyecto tiene como objetivo principal la creación de un dataset clasificado a partir de un vídeo, enfocándose en la identificación precisa de personas como “persona objetivo” y “personas no objetivo”. A continuación se desarrollan los objetivos específicos detalladamente:

- ★ **Extracción y guardado en disco:** Implementar un proceso automatizado para la extracción de fotogramas del vídeo y su posterior almacenamiento en el disco utilizando la librería OpenCV en Python. Este proceso asegurará que cada fotograma del vídeo se capture y guarde como una imagen individual, preparándolos para la etapa de detección y clasificación.
- ★ **Detección de personas en fotogramas (YOLO):** Utilizar la red neuronal YOLO (You Only Look Once) para la detección automática de personas en los fotogramas del vídeo. YOLO es elegida por su capacidad de realizar detecciones rápidas y precisas con una sola pasada por la red neuronal, lo que es crucial para procesar los fotogramas en tiempo real o cerca de tiempo real.
- ★ **Clasificación automática del dataset:** Desarrollar un algoritmo automático que clasifique los fotogramas del vídeo en dos categorías: “persona objetivo” y “personas no objetivo”. Esta clasificación se basará en la comparación del Error

3.3. FASE 3. GENERACIÓN Y CLASIFICACIÓN DE DATASET

Cuadrático Medio (MSE) entre la imagen de color en escala de grises de dos ROIs consecutivos (La imagen del objetivo anterior y los ROIs del fotograma actual). Si la diferencia es menor que el criterio establecido (por ejemplo, $MSE < 70$), el fotograma será etiquetado automáticamente como “persona objetivo”.

- ★ **Clasificación manual del dataset:** Implementar una interfaz de usuario interactiva que permita al usuario intervenir en la clasificación de los fotogramas cuando el método automático no pueda decidir de manera clara. La interfaz permitirá al usuario asignar la etiqueta de “persona objetivo” o “personas no objetivo” utilizando controles específicos, garantizando así una clasificación precisa y ajustada a criterios específicos del proyecto.
- ★ **Interfaz de usuario:** Desarrollar una interfaz gráfica amigable que facilite la interacción del usuario para la clasificación manual de los fotogramas y la gestión de teclas utilizando la librería OpenCV. Esta interfaz será diseñada para proporcionar una visualización clara de los fotogramas, resultados tras clasificación y controles intuitivos para que el usuario pueda marcar la clasificación deseada de manera eficiente y precisa.

Estos objetivos están diseñados para asegurar la precisión, eficiencia y robustez del proceso de generación del dataset clasificado, fundamental para las fases posteriores del proyecto, como el entrenamiento de redes neuronales para la detección de personas objetivo.

3.3.3. Explicaciones técnicas

En esta sección se detallarán las especificaciones técnicas clave utilizadas en el proyecto para la generación del dataset clasificado y la detección de personas en los fotogramas de vídeo. A continuación, se explica cada elemento relevante:

- **MSE (Error Cuadrático Medio):** El MSE (Mean Squared Error) es una métrica utilizada para medir la diferencia entre dos imágenes en escala de grises de igual

tamaño. Cuanto menor sea el valor del MSE, mayor será la similitud entre las dos imágenes comparadas en términos de intensidades de gris.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (3.1)$$

- **Red neuronal YOLO (You Only Look Once):** es un modelo de detección de objetos en imágenes que permite la detección en tiempo real con una sola pasada a través de la red neuronal. Funciona dividiendo la imagen en una cuadrícula y prediciendo cajas delimitadoras y probabilidades de clasificación para cada celda. Esto permite una detección rápida y precisa de objetos, incluidas las personas, en las imágenes procesadas.
- **Límites de precisión y MSE:** se establece una precisión mínima del 80 % para las detecciones realizadas por YOLO y un valor de MSE máximo de 70.0 para asegurar la calidad y confiabilidad de las clasificaciones. Esta elección se basa en la necesidad de evitar falsos positivos y negativos significativos durante la identificación de la persona objetivo en los fotogramas del vídeo.

Las condiciones que se tienen en cuenta en esta fase son las siguientes:

- ★ **Precisión de la detección:** Se considera un umbral adecuado un 80 % (0.8) para equilibrar la capacidad del modelo de detectar correctamente a la persona objetivo con un bajo índice de errores. Una precisión más alta, como del 90 %, podría limitar la capacidad del modelo para detectar la persona objetivo si esta se aleja, ya que las detecciones podrían no cumplir con el umbral de confianza establecido. Por otro lado, reducir la precisión por debajo del valor predefinido aumentaría el riesgo de que se produzcan identificaciones incorrectas.
- ★ **MSE entre ROIs:** El valor del MSE máximo de 70.0 asegura que las diferencias entre las imágenes comparadas no sean significativamente altas, lo que podría indicar cambios demasiado drásticos en la apariencia de la

persona objetivo entre fotogramas. Esto ayuda a mantener la coherencia en la identificación de la persona objetivo a lo largo del vídeo.

Estas especificaciones técnicas, combinadas con el uso de herramientas como Python, OpenCV para manipulación de imágenes y vídeos, y el uso de NUC con especificaciones adecuadas (como se detalló anteriormente), garantizan que el dataset resultante sea de alta calidad y adecuado para el entrenamiento efectivo de la red neuronal en fases posteriores del proyecto.

3.3.4. Librerías principales

En esta fase se utilizarán varias librerías de Python para cumplir con los objetivos establecidos. A continuación se detallan las librerías principales y su papel específico en el proceso:

- ★ **PyRealSense2[10]**: Se utilizará para la lectura de vídeos almacenados en disco. Esta librería permite acceder a datos de profundidad y color grabados previamente con cámaras Intel RealSense. `pyrealsense2` facilita la conexión con los archivos de grabación y la extracción de fotogramas, lo cual es esencial para procesar el vídeo en busca de regiones de interés (ROIs) que contienen personas.

Para su instalación se debe seguir el siguiente comando:

```
# Instalación de la librería PyRealSense2
pip3 install pyrealsense2
```

- ★ **OpenCV[11]**: Es una librería de procesamiento de imágenes y visión por computadora ampliamente utilizada. En este proyecto, se empleará OpenCV para varias tareas clave: la extracción de fotogramas del vídeo para su procesamiento, el guardado de los fotogramas extraídos como archivos de imagen en el disco, utilizando funciones como `cv2.imwrite`, y la creación de una interfaz visual que permita la interacción del usuario durante la clasificación manual de fotogramas, mostrando imágenes y capturando entradas del teclado.

Para su instalación se debe seguir el siguiente comando:

```
# Instalación de la librería OpenCV  
pip3 install opencv-python
```

- ★ **Ultralytics (YOLO)[16]**: se utilizará específicamente para cargar y utilizar el modelo YOLO (You Only Look Once) en la detección de personas. YOLO es un modelo de red neuronal convolucional que se destaca por su capacidad de realizar detecciones rápidas y precisas. En este proyecto, YOLO se empleará para cargar el modelo preentrenado de YOLO para detectar personas en los fotogramas y para identificar y delimitar las regiones de interés (ROIs) que contienen personas en los fotogramas extraídos. YOLO proporciona una alta precisión en la detección, lo que es crucial para asegurar que solo se consideren detecciones con una precisión superior al 80%.

Para su instalación se debe seguir el siguiente comando:

```
# Instalación de la librería ultralytics (Contiene YOLO)  
pip3 install ultralytics
```

Estas librerías, en conjunto, permiten implementar un flujo de trabajo eficiente y robusto para la creación del dataset clasificado, desde la extracción de fotogramas hasta la detección y clasificación de personas.

3.3.5. Desarrollo del código

En esta sección se detalla el desarrollo del código utilizado para la generación del dataset clasificado. El proyecto se divide en dos componentes de Python distintos: uno para la extracción y almacenamiento de fotogramas del vídeo, y otro para la detección y clasificación de personas en los fotogramas.

Primer código

En este apartado, se exploran los métodos clave de la clase encargada de extraer y guardar en disco todos los fotogramas de un vídeo pregrabado. Se detalla el proceso para leer el vídeo, extraer cada fotograma y almacenar estos fotogramas individualmente en el disco. Se puede consultar la implementación completa de esta clase en el repositorio de GitHub[17].

1. **compute() [65-86]:** Se ejecuta periódicamente para procesar los fotogramas obtenidos de una cámara o un archivo .bag. Primero, intenta obtener nuevos fotogramas. Si están disponibles, los procesa para extraer los fotogramas de color y de profundidad. Luego, muestra estos fotogramas en la interfaz de usuario utilizando OpenCV. Si está habilitada la opción de previsualización, también guarda los fotogramas en el disco. Finalmente, incrementa el contador de fotogramas. Si no hay más fotogramas disponibles, genera un archivo JSON con los resultados y finaliza la ejecución del programa.
2. **comprobacion_requisitos_minimos() [90-122]:** Verifica que se cumplan las condiciones necesarias para que el programa funcione correctamente. Realiza las siguientes acciones:
 - ★ **Verificación del archivo de grabación:** Comprueba que el archivo de grabación existe y tiene la extensión “.bag”. Si no se cumplen estas condiciones, el programa termina con un mensaje de error.
 - ★ **Verificación del directorio de destino de fotogramas:** Si el directorio ya existe y está habilitado el reemplazo, elimina el directorio existente y lo recrea junto con subdirectorios para fotogramas de color y profundidad. Si el reemplazo no está habilitado, el programa termina la ejecución para evitar errores.
 - ★ **Verificación del directorio de destino de fotogramas:** Si no se está en modo de previsualización, verifica que el directorio de destino para guardar

los fotogramas exista. Si no existe, termina la ejecución con un mensaje de error.

3. **iniciar_conexion_con_grabacion()** [126-146]: Establece la conexión para procesar los datos de grabación. Primero, se configura la conexión para recibir la información desde un archivo de grabación, sin activar el playback. Luego, se habilitan todos los canales de datos disponibles, como el de color y el de profundidad. A continuación, se inicia el flujo de datos utilizando la configuración definida previamente. Finalmente, se registra el tiempo de inicio de la ejecución y se imprime un mensaje indicando que la conexión y el flujo de datos se han activado correctamente.
4. **guardar_frames_en_disco()** [195-208]: se encarga de almacenar en el disco los fotogramas de color y de profundidad procesados. Para ello, primero construye las rutas donde se guardarán los fotogramas en función del número de fotograma actual. Luego, guarda cada fotograma en su respectiva ubicación en el disco, utilizando formatos de imagen adecuados (como “*jpeg*”). Este método asegura que cada fotograma se guarde correctamente para su posterior análisis o uso.
5. **interfaz_usuario()** [163-174]: Se encarga de mostrar los fotogramas de color y de profundidad al usuario en tiempo real utilizando la interfaz de OpenCV. Primero, presenta los fotogramas en ventanas separadas, una para el fotograma de color y otra para el de profundidad. Luego, establece un tiempo de espera mínimo de 1 milisegundo para permitir la fluidez en la visualización. Por último, controla si se ha presionado alguna tecla, permitiendo que el usuario interactúe con la visualización y, por ejemplo, detener el proceso si se presiona la tecla correspondiente. Es similar a la que se aprecia en la figura 3.9
6. **generacion_de_resultados()** [212-236]: Realiza el cálculo y la generación de un resumen de los resultados al finalizar el procesamiento. Primero, calcula el tiempo total de ejecución. Luego, crea un diccionario con la información relevante, como el tiempo de conexión, el número total de fotogramas guardados

y la ruta destino de la grabación. Imprime estos resultados en la consola para revisión temporal. También guarda esta información en un archivo JSON en la ubicación especificada, si el directorio para el archivo existe; de lo contrario, emite una advertencia.

Segundo código

En este apartado, se desglosan los métodos clave de la clase que clasifica los fotogramas almacenados en categorías de “persona objetivo” y “no objetivo”, con el fin de crear un dataset útil. Se explica cómo se realiza la clasificación automática de los fotogramas y cómo se organiza la información para facilitar el uso en el dataset. Se puede revisar la implementación detallada de esta clase en el repositorio de GitHub[18].

1. **compute() [74-111]**: gestiona el procesamiento de fotogramas en un bucle hasta que todos los fotogramas disponibles han sido procesados. Primero, comprueba si el contador de fotogramas es menor que el número total de fotogramas disponibles. Si hay fotogramas pendientes, carga el fotograma actual desde la ruta especificada y lo procesa utilizando la red neuronal YOLO para detectar objetos y obtener precisiones de las detecciones.

Luego, el método filtra los resultados para obtener las cajas de colisiones (es decir, las áreas en las que se han detectado objetos) y las precisiones de detección correspondientes. Si no se ha encontrado un objetivo de manera automática en fotogramas anteriores, el método permite al usuario seleccionar manualmente el objetivo en el fotograma actual.

Después de procesar el fotograma, guarda las regiones de interés (ROIs) clasificadas, es decir, las partes del fotograma donde se han detectado objetos, en el directorio correspondiente. También actualiza la interfaz de usuario para mostrar el fotograma original y el fotograma con las detecciones.

Finalmente, incrementa el contador de fotogramas y, si todos los fotogramas han sido procesados, calcula y guarda un resumen del procesamiento.

Posteriormente, el programa termina con un mensaje indicando el fin de la ejecución.

2. **comprobacion_requisitos_minimos()** [115-142]: Asegura que se cumplen las condiciones básicas necesarias para el correcto funcionamiento del programa. Primero, verifica que el directorio que contiene el dataset sin clasificar existe. Si no es así, el programa se detiene con un mensaje de error.

Luego, confirma que el directorio padre del destino del dataset clasificado también existe. Si este directorio no está presente, el programa se detiene con un mensaje de error.

El siguiente paso es comprobar si el reemplazo del dataset existente está habilitado. Si el reemplazo no está permitido, el programa se detiene con un mensaje de error para evitar posibles conflictos.

A continuación, actualiza la ruta del directorio de destino del dataset clasificado para incluir dos subdirectorios para guardar los ROIs de la persona objetivo y de las que no lo son. Si estos subdirectorios ya existen, los elimina y lo recrea para garantizar que esté vacío antes de comenzar el procesamiento.

3. **resultados_filtrados()** [160-175]: procesa los resultados obtenidos de una red neuronal para filtrar las detecciones relevantes en una imagen. Primero, crea dos listas vacías: una para almacenar las cajas de colisión (es decir, las coordenadas de las áreas detectadas) y otra para guardar la precisión de cada detección.

A continuación, itera sobre cada detección en los resultados de la red neuronal. Para cada detección, verifica si se trata de una persona y si la precisión de la detección supera un umbral mínimo predefinido.

Si ambas condiciones se cumplen, añade la caja de colisión a la lista de cajas y la precisión a la lista de precisiones. La caja de colisión se convierte en una lista de coordenadas enteras, y la precisión se convierte en un número flotante.

Al final del método, devuelve las dos listas: una con las cajas de colisión y otra con las precisiones de las detecciones filtradas.

4. **obtencion_indice_objetivo_manual()** [179-201]: permite al usuario seleccionar manualmente el objetivo de interés en un fotograma de vídeo.

Primero, muestra el fotograma en pantalla con la región de interés (ROI) correspondiente a cada caja de colisión detectada por el modelo. Para cada región de interés, se muestra una ventana con la imagen recortada. El usuario debe indicar si la región mostrada contiene al objetivo deseado.

Si el usuario presiona la tecla “ENTER”, se considera que ha encontrado al objetivo, y el índice de esa región se guarda como el objetivo. Si se presiona cualquier otra tecla, se pasa a la siguiente región.

Al final del proceso, se cierra la ventana mostrada y se devuelve el índice de la región de interés que el usuario ha identificado como el objetivo. Si no se ha encontrado un objetivo, se devuelve -1.

5. **obtencion_indice_objetivo_automatico()** [205-244]: Se encarga de identificar automáticamente la región de interés que más probablemente corresponde al objetivo en un fotograma, utilizando un criterio de similitud basado en la comparación de imágenes.

Cada región de interés se redimensiona y convierte a escala de grises para facilitar la comparación. La región de interés actual se compara con la región de interés del fotograma anterior, calculando el error cuadrático medio (MSE) entre ambas. Este MSE mide cuán similares son las dos regiones; un MSE más bajo indica una mayor similitud.

Si el MSE de una región actual es menor que el MSE de la mejor región encontrada hasta ahora y también es menor que el valor máximo aceptable definido, se actualiza la mejor coincidencia con la región actual.

Si al final del proceso no se encuentra una coincidencia adecuada (es decir, el índice del objetivo sigue siendo -1), el método recurre al proceso manual de selección para asegurar que se identifica al objetivo correctamente.

6. **guardado_rois_clasificados()** [248-274]: se encarga de guardar en disco las regiones de interés (ROIs) clasificadas en función de si corresponden al objetivo identificado o no.

Primero, se inicializa un contador de detecciones. Luego, para cada caja de colisión detectada en el fotograma, se extrae la región de interés correspondiente. Dependiendo de si la caja de colisión actual corresponde al objetivo identificado o no, se determina la ruta de destino para guardar la imagen de la región de interés.

Si la región de interés corresponde al objetivo, se guarda en el directorio destinado a las imágenes del objetivo. La región de interés del objetivo también se almacena como la región anterior para futuras comparaciones. Para las demás regiones de interés, se guarda en el directorio destinado a las imágenes de no objetivo.

Cada región de interés se guarda con un nombre que incluye el número del fotograma y el índice de la detección. Después de procesar todas las cajas de colisión, se incrementa el contador de detecciones.

Este método organiza y guarda las imágenes de las regiones detectadas en carpetas separadas según su clasificación, lo que facilita la gestión y el análisis posterior de los datos.

7. **interfaz_usuario()** [278-316]: Se encarga de mostrar el fotograma original y el fotograma con las detecciones en una interfaz visual, similar a la que se aprecia en la figura 4.1. Aquí están los pasos que sigue:

★ **Dibujo de cajas de colisión:** Primero, se crea una copia del fotograma original para aplicar los resultados. Para cada detección en la lista, el método dibuja un rectángulo alrededor de la región de interés en el fotograma copiado. Si la detección corresponde a la persona objetivo, se usa un color verde; de lo contrario, se usa rojo.

- ★ **Adición de etiquetas:** Se añaden etiquetas de texto al fotograma con las detecciones. La etiqueta indica si la persona es el objetivo o no y muestra la precisión de la detección.
- ★ **Visualización:** Se muestran dos imágenes usando OpenCV, una con el fotograma original y otra con las detecciones. El fotograma original se muestra tal cual, mientras que el fotograma con detecciones incluye las bounding boxes y las etiquetas.
- ★ **Control de fluidez:** Se asegura de que la visualización sea fluida al asignar un tiempo de espera mínimo de 1 milisegundo entre cada fotograma mostrado. Además, se utiliza un método para manejar la entrada del teclado, permitiendo interacciones como la detención del proceso con la tecla ESC o la selección de un objetivo con la tecla ENTER.

Este método es crucial para que el usuario pueda verificar visualmente las detecciones realizadas por el sistema y hacer ajustes si es necesario.

8. **generacion_de_resultados()** [344-367]: Calcula el tiempo total de ejecución del proceso y crea un diccionario con el tiempo de ejecución, el número de fotogramas guardados y la ruta de destino de los fotogramas clasificados. Luego, imprime estos resultados en la consola y guarda el diccionario en un archivo JSON si la ruta del archivo existe.

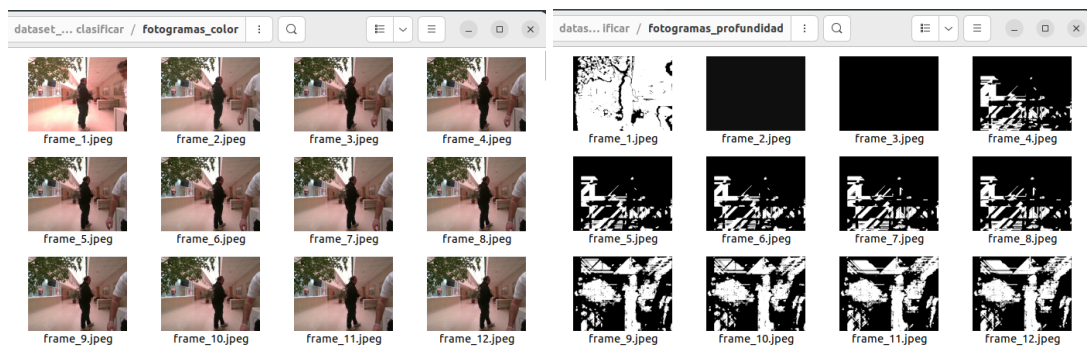
3.3.6. Resultados

En este apartado, se detallarán los resultados obtenidos del funcionamiento del componente. Los resultados se presentan de manera separada para cada uno de los códigos implementados.

Primer código

El primer código ejecuta el procesamiento de un vídeo pregrabado desde una cámara Intel RealSense y genera los siguientes resultados finales:

- ★ **Generación de carpeta con fotogramas:** Se crea una carpeta en el directorio especificado para almacenar los fotogramas del vídeo pregrabado. Dentro de esta carpeta, se generan dos subdirectorios: uno para los fotogramas de color como el que se muestra en la figura 3.12a y otro para los fotogramas de profundidad, figura 3.12b. Cada fotograma se guarda en formato JPEG, numerado secuencialmente para mantener un orden claro.



(a) Carpeta con fotogramas de color. (b) Carpeta con fotogramas de profundidad.

Figura 3.12: Formato carpetas de fotogramas.

- ★ **Salidas por consola:** Al finalizar el procesamiento, el programa muestra los siguientes datos clave en la consola: tiempo de ejecución, número de fotogramas guardados y ruta destino de los fotogramas.
- ★ **Generación de archivo JSON:** Además de las salidas por consola, se crea un archivo JSON (data.json) que almacena de forma estructurada los datos clave del procesamiento: tiempo de ejecución, número de fotogramas guardados y ruta destino de los fotogramas. Este archivo JSON proporciona un registro permanente y estructurado de los resultados, útil para referencias futuras.

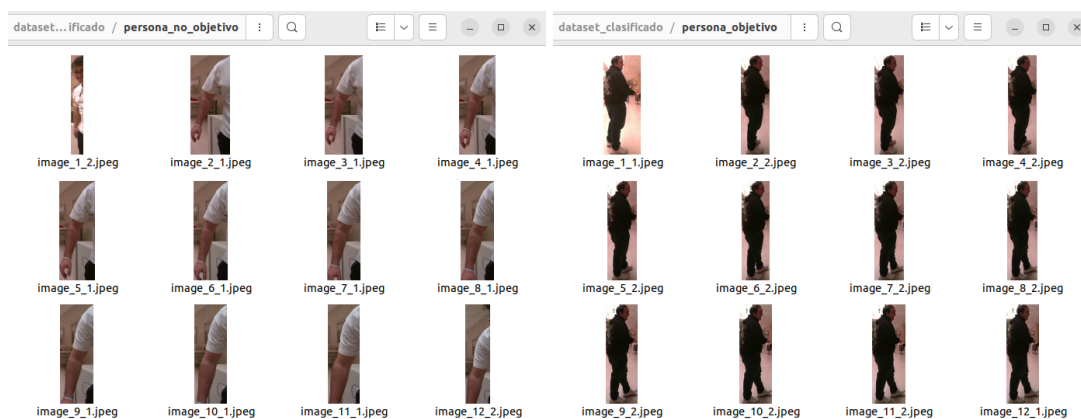
```
1{"Tiempo de ejecucion" : 140.012,
2 "Numero de frames guardados" : 6907,
3 "Ruta destino de los fotogramas" : /media/robocomp/data_tfg/dataset_sin_clasificar}
```

Figura 3.13: Ejemplo del archivo JSON generado.

Segundo código

El segundo código se centra en la clasificación de fotogramas de un dataset utilizando la red neuronal YOLO para identificar y guardar imágenes de una “persona objetivo” y de otras personas.

- ★ **Generación de carpeta con fotogramas clasificados:** Se genera una carpeta denominada “*dataset_clasificado*” en la ruta especificada. Dentro de esta carpeta, se crean dos subcarpetas como se muestran en las figuras 3.14a y 3.14b. Cada fotograma se guarda en una de estas subcarpetas en formato JPEG, con nombres que indican el número de fotograma y detección.



(a) Carpeta con ROIs no objetivos.

(b) Carpeta con ROIs del objetivo.

Figura 3.14: Formato carpetas de ROIs.

- ★ **Salidas por consola:** Durante la ejecución, se muestra información clave en la consola. Esto incluye el inicio de una nueva iteración, avisos sobre el estado del procesamiento, y finalmente, un resumen de los resultados: tiempo de ejecución total, número de fotogramas guardados, y la ruta destino de los fotogramas clasificados.
- ★ **Generación de archivo JSON:** Al finalizar el procesamiento, se genera un archivo JSON (data.json) que almacena los datos clave de la ejecución: tiempo de ejecución, número de fotogramas guardados, y la ruta destino de los

fotogramas clasificados. Este archivo proporciona un registro estructurado de los resultados, útil para referencias futuras.

```
1{"Tiempo de ejecucion" : 400.359,  
2 "Numero de fotogramas procesados" : 6907,  
3 "Ruta destino del dataset clasificado" : /media/robocomp/data_tfg/dataset_clasificado}
```

Figura 3.15: Ejemplo del archivo JSON generado.

3.3.7. Conclusiones

El primer código es perfecto en el sentido de que realiza su trabajo a la perfección. Genera una carpeta con los fotogramas del vídeo pregrabado, tanto en color como en profundidad, y proporciona salidas por consola y en un archivo JSON en disco que muestran datos clave, como el número de fotogramas, el tiempo de ejecución y el destino de los fotogramas.

Con respecto al segundo código, aunque requiere intervención del usuario para identificar a la “persona objetivo” en el primer fotograma, esta intervención es prácticamente mínima. Además de esta identificación inicial, puede ser necesario que el usuario intervenga en las siguientes situaciones:

- ★ Si se pierde el objetivo, lo cual no debería ocurrir si el proceso se ejecuta correctamente. El código necesitará que el usuario vuelva a identificar al objetivo.
- ★ Si el objetivo se aleja demasiado del campo de visión de la cámara, lo que podría reducir la precisión de la detección por debajo del 80 %, provocando una nueva solicitud de identificación manual.
- ★ Si hay problemas durante la transmisión de los fotogramas, especialmente en la primera fase, debido a una conectividad baja que cause diferencias de tiempo significativas entre fotogramas. Esto podría resultar en que las diferencias entre las dos personas objetivo (la del fotograma anterior y la del fotograma actual) no cumplan con el máximo permitido para el MSE de 70.

3.3. FASE 3. GENERACIÓN Y CLASIFICACIÓN DE DATASET

Sin embargo, en lo que respecta al proyecto, el proceso es automático en un 98/99%, lo que garantiza una gran eficiencia y precisión en la clasificación y procesamiento de los fotogramas.

3.3.8. Ampliaciones a futuro

Con el objetivo de potenciar y enriquecer aún más la funcionalidad y eficacia de los componentes desarrollados, se han identificado diversas áreas para futuras mejoras y expansiones. Estas ampliaciones no solo apuntan a fortalecer la capacidad actual de los sistemas implementados, sino también a explorar nuevas posibilidades tecnológicas y algoritmos avanzados en el ámbito del procesamiento de vídeo y visión por computadora. A continuación, se detallan las áreas clave que podrían beneficiarse de futuros desarrollos y optimizaciones.

Primer código

- ★ **Mejora de la interfaz gráfica:** Implementar una interfaz gráfica más intuitiva y amigable para el usuario, que permita una mejor visualización y manejo de las funciones de lectura y procesamiento de vídeo.
- ★ **Soporte para múltiples formatos de vídeo:** Ampliar la funcionalidad para admitir la captura y procesamiento de múltiples formatos de vídeo, no limitándose únicamente a archivos “.bag”.
- ★ **Optimización de recursos:** Realizar una optimización más profunda de los recursos del sistema durante el procesamiento de vídeo, considerando técnicas como el uso de GPU para acelerar algunos procesos. También se podría utilizar el lenguaje C++ en lugar Python para aumentar la eficiencia.
- ★ **Integración de funciones de post-procesamiento:** Incluir funciones de post-procesamiento de vídeo, como la corrección de color automática, para mejorar la calidad visual de los resultados finales.

Segundo código

- ★ **Mejora de la detección automática de personas:** Refinar el algoritmo de detección automática de personas para aumentar la precisión y reducir la dependencia de umbrales predefinidos.
- ★ **Manejo automático de situaciones problemáticas:** Implementar mecanismos automáticos para manejar situaciones problemáticas como la pérdida temporal del objetivo debido a obstrucciones o cambios abruptos en el entorno.
- ★ **Integración de feedback visual en tiempo real:** Incluir una retroalimentación visual en tiempo real para informar al usuario sobre el estado del sistema, como la posición actual del objetivo y la calidad de la detección en cada fotograma.
- ★ **Optimización de recursos:** Mejorar la eficiencia del código durante el procesamiento de fotogramas, considerando técnicas como el uso de GPU para acelerar la detección de objetos con redes neuronales o el procesamiento de dichos objeto (ROIs de personas). También se podría utilizar el lenguaje C++ en lugar Python para aumentar la eficiencia ya que es un lenguaje más rápido pero menos legible.

3.4. Fase 4. Entrenamiento de la red neuronal

3.4.1. Introducción

El presente proyecto tiene como objetivo aplicar la inteligencia artificial en tareas cotidianas, específicamente en la identificación de personas objetivo dentro de un fotograma individual de un video. Este desarrollo combina técnicas avanzadas de visión artificial con redes neuronales profundas, enfocándose en la automatización de tareas simples pero repetitivas.

En esta fase del proyecto, se entrenará una red neuronal para identificar a la persona objetivo entre todas las personas presentes en un fotograma. Se ha seleccionado la

3.4. FASE 4. ENTRENAMIENTO DE LA RED NEURONAL

arquitectura de red neuronal “*ResNet18*” debido a su simplicidad y facilidad de uso, y se entrenará desde cero, sin utilizar pesos preentrenados. El entrenamiento se llevará a cabo utilizando el framework RoboComp y la librería PyTorch, integrando así el resultado como un componente del mencionado framework.

Para el entrenamiento, se dispone de un conjunto de datos previamente recopilado y clasificado, compuesto por imágenes extraídas de videos. Estas imágenes contienen regiones de interés que incluyen personas. Cada imagen está etiquetada con un valor numérico entre 0 y 1, donde 1 indica la persona objetivo y 0 cualquier otra persona.

El objetivo principal de esta fase es alcanzar un rendimiento aceptable en la identificación de la persona objetivo, con una tasa de error inferior al 1 %. La métrica utilizada para evaluar el rendimiento de la red neuronal será la diferencia entre el valor real y el valor predicho.

La implementación de la red neuronal entrenada se utilizará en la siguiente fase del proyecto para desarrollar un sistema de detección en tiempo real del usuario objetivo. Este sistema podría tener el potencial de ser aplicado en diversos escenarios, permitiendo el seguimiento de una persona concreta con precisión y eficiencia.

3.4.2. Objetivos

En esta fase del proyecto, se delinean una serie de objetivos claros y específicos que guiarán el desarrollo y entrenamiento de la red neuronal para la identificación de la persona objetivo dentro de un fotograma de video. Estos objetivos están diseñados para asegurar que se alcancen los resultados deseados con una alta precisión y eficiencia. A continuación, se detallan los objetivos de esta fase:

- ★ **Carga de dataset clasificado:** Es esencial para entrenar la red neuronal, ya que proporciona los datos necesarios para que el modelo aprenda a identificar a la persona objetivo. Este dataset, que contiene imágenes etiquetadas, se cargará mediante la utilización de rutas de acceso en lugar de almacenar las imágenes directamente en memoria. Este enfoque optimiza el uso de recursos y aumenta

la eficiencia del proceso, permitiendo cargar las imágenes solo cuando sean necesarias.

- ★ **Entrenamiento del modelo:** El objetivo es entrenar una red neuronal “*ResNet18*” utilizando el optimizador “*SGD*” con una tasa de aprendizaje de 0,001 y la función de pérdida “*MSELoss*”. Las imágenes se cargarán en lotes, se convertirán a tensores y se asignarán al dispositivo adecuado. El entrenamiento se realizará durante 10 épocas en lotes de 32 y dimensiones de imagen de 350 x 150 píxeles. Este proceso está diseñado para optimizar el rendimiento del modelo en la identificación de la persona objetivo.
- ★ **Muestra de los resultados por épocas:** Se registrarán y mostrarán las pérdidas de entrenamiento y validación por época. Los resultados se almacenarán en listas y se graficarán al finalizar el entrenamiento usando Matplotlib. Los resultados se actualizarán después de cada época y deben mostrar una disminución en las métricas de pérdida, lo que indica que la red neuronal está aprendiendo correctamente.
- ★ **Guardado de los parámetros de la red neuronal:** Este objetivo consiste en guardar los parámetros de la red neuronal al finalizar el entrenamiento. Los parámetros se almacenarán en archivos con extensión “.pth” en disco utilizando PyTorch. Esto permite cargar la red neuronal preentrenada para su uso posterior. Para evitar la sobrescritura, se implementará un flag dedicado para ello.

3.4.3. Explicaciones técnicas

- ★ **Parámetros de entrenamiento de red neuronal:** Estos parámetros son esenciales para optimizar el modelo y mejorar su capacidad de identificar a la persona objetivo en imágenes. A continuación, se describen brevemente:
 - **Número de épocas:** Define cuántas veces el modelo pasará por el conjunto completo de datos de entrenamiento.

- **Tasa de aprendizaje:** Controla el tamaño de los pasos que da el optimizador durante la actualización de los parámetros del modelo.
 - **Tamaño del lote:** Es el número de muestras que se procesan antes de actualizar el modelo.
 - **Optimizador (SGD):** Es una técnica para optimizar la función de pérdida del modelo ajustando iterativamente los parámetros en función del gradiente de la función de pérdida.
 - **Función de pérdida (MSELoss):** Mide la media de los cuadrados de los errores entre los valores predichos por el modelo y los valores reales.
 - **Dimensiones de entrada:** Define el tamaño y la estructura de las imágenes de entrada al modelo.
- ★ **Funcionamiento de la red neuronal:** La red neuronal procesa la información a través de múltiples capas, ajustando sus pesos para minimizar el error entre las predicciones y los resultados deseados. Utiliza funciones de activación para introducir no linealidad y técnicas de optimización para mejorar su precisión durante el entrenamiento. Una vez entrenada, la red aplica estos ajustes para realizar predicciones en nuevos datos.
- **Entrenamiento:** Sirve para ajustar los parámetros del modelo (como los pesos y sesgos) de manera que el modelo pueda aprender de los datos y realizar predicciones más precisas. Durante el entrenamiento, el modelo se expone a un conjunto de datos y ajusta sus parámetros para minimizar el error en sus predicciones, lo que le permite “aprender” las características importantes de los datos.
 - **Validación:** Evalúa cómo de bien está aprendiendo el modelo utilizando un conjunto de datos diferente al que se utilizó para el entrenamiento. Sirve para verificar la capacidad del modelo para generalizar a nuevos datos y detectar problemas como el sobreajuste (donde el modelo se adapta demasiado a los datos de entrenamiento y no funciona bien en datos

nuevos). La validación ayuda a ajustar el entrenamiento y a seleccionar el mejor modelo.

- ★ **Significado de las pérdidas:** Las pérdidas en el proceso de entrenamiento y validación representan la discrepancia entre las predicciones de la red neuronal y los valores reales. Una pérdida alta indica un error significativo y sugiere que el modelo necesita mejorar. A medida que la red se entrena, se espera que la pérdida disminuya, reflejando un ajuste más preciso a los datos y una mejor capacidad para generalizar a nuevas entradas.
- ★ **Guardado de los parámetros de la red neuronal:** Permite preservar los pesos y biases del modelo entrenado, facilitando su uso futuro sin necesidad de reentrenar. Esto se realiza guardando estos parámetros en un archivo con extensión específica (como .pth en PyTorch) al finalizar el entrenamiento. Este archivo permite cargar el modelo más tarde para realizar predicciones o continuar el entrenamiento, asegurando así la reproducibilidad y eficiencia en el uso del modelo.

3.4.4. Librerías principales

En esta fase se utilizarán varias librerías de Python para cumplir con los objetivos establecidos. A continuación se detallan las librerías principales y su papel específico en el proceso:

- ★ **PyTorch[19]:** Se utiliza para gestionar las transformaciones de datos, crear y manejar los conjuntos de datos y cargadores de datos, y facilitar el entrenamiento de la red neuronal. También se encarga del cálculo de la pérdida y la actualización de los parámetros de la red, así como del almacenamiento del modelo entrenado en el disco.

```
# Instalación de la librería PyTorch
pip3 install torch torchvision
```

3.4. FASE 4. ENTRENAMIENTO DE LA RED NEURONAL

- ★ **Matplotlib[20]**: Se utiliza para generar gráficos de las pérdidas de entrenamiento y validación a lo largo de las épocas. Esto permite visualizar el rendimiento de la red neuronal durante el proceso de entrenamiento, guardando estos gráficos en el disco para un análisis posterior.

```
# Instalación de la librería matplotlib  
pip3 install matplotlib
```

- ★ **OpenCV[11]**: Se utiliza para leer y procesar las imágenes del dataset. Se emplea para cargar las imágenes desde las rutas especificadas, redimensionarlas al tamaño adecuado y prepararlas para ser transformadas y utilizadas en el entrenamiento de la red neuronal.

```
# Instalación de la librería OpenCV  
pip3 install opencv-python
```

En resumen, las librerías PyTorch, Matplotlib y OpenCV son fundamentales en este proyecto. PyTorch se utiliza para el manejo de la red neuronal, Matplotlib facilita la visualización de los resultados y OpenCV se encarga del procesamiento de imágenes. Estas herramientas en conjunto aseguran un flujo de trabajo eficiente y efectivo en el desarrollo y evaluación del modelo.

3.4.5. Desarrollo del código

En este apartado, se detallan los métodos clave de la clase que entrena una red neuronal para detectar a la persona objetivo en una multitud, utilizando el dataset clasificado creado anteriormente. Se explica cómo se prepara el dataset, cómo se configura la red neuronal y cómo se evalúa su rendimiento. Se puede encontrar la implementación completa de esta clase en el repositorio de GitHub[21].

1. **compute() [142-180]**: Gestiona el proceso de entrenamiento y validación de una red neuronal, alternando entre ambos según sea necesario. Durante el

entrenamiento, se toman lotes de imágenes y se utilizan para ajustar los parámetros de la red, buscando minimizar la diferencia entre las predicciones de la red y los valores reales. A medida que avanza el entrenamiento, el método acumula las pérdidas asociadas y, cuando se han procesado suficientes imágenes, cambia a la fase de validación.

En la fase de validación, el método evalúa el rendimiento de la red utilizando un conjunto separado de datos para verificar cuánto se ajusta el modelo a los datos que no ha visto durante el entrenamiento. Aquí también se acumulan las pérdidas para poder compararlas con las pérdidas del entrenamiento.

Una vez que se ha completado la validación, se vuelve a la fase de entrenamiento, incrementando el número de épocas completadas. Si se han alcanzado el número total de épocas programadas, el método finaliza el proceso, guarda los resultados y genera gráficos que resumen el desempeño del modelo a lo largo del tiempo.

2. **comprobacion_requisitos_minimos()** [184-200]: Verifica que se cumplan ciertas condiciones necesarias antes de que comience el entrenamiento de la red neuronal. Primero, se asegura de que exista la carpeta que contiene el conjunto de datos, deteniendo el programa si no la encuentra. Luego, verifica que el porcentaje asignado para la validación de datos esté dentro de un rango válido, es decir, entre 0 y 1.

También se asegura de que la ruta donde se guardará el modelo entrenado exista, y si no, el programa se detiene. Por último, si no se ha habilitado el reemplazo del modelo y ya existe un modelo guardado en la ruta de destino, el método detiene el programa para evitar que el modelo anterior sea sobrescrito accidentalmente. Estas verificaciones aseguran que el entorno esté correctamente configurado antes de iniciar el proceso de entrenamiento.

3. **preparacion_entorno()** [204-227]: se encarga de configurar todo lo necesario para comenzar el entrenamiento de la red neuronal. Primero, carga los datos dividiéndolos en conjuntos de entrenamiento y validación, según un porcentaje

especificado. Luego, crea conjuntos de datos personalizados a partir de estas divisiones y los encapsula en objetos que facilitan su manejo en lotes.

Después, se inicializa la red neuronal, seleccionando una variante de ResNet para regresión. La red se configura para que funcione en el dispositivo adecuado, como una GPU si está disponible. Finalmente, el método establece la función de pérdida, que mide la discrepancia entre las predicciones de la red y los valores reales, y configura un optimizador que ajustará los parámetros de la red durante el entrenamiento. Con todo esto, el entorno queda preparado para iniciar el proceso de entrenamiento y validación.

4. **cargar_dataset()** [231-263]: Se encarga de preparar los datos de entrada para el entrenamiento y la validación de la red neuronal. Primero, recopila todas las rutas de las imágenes que están clasificadas en dos categorías: “objetivo” y “no objetivo”. A partir de estas rutas, crea dos listas: una con las imágenes objetivo y otra con las imágenes no objetivo, asignando etiquetas correspondientes (1 para objetivo y 0 para no objetivo).

Luego, el método combina las listas de imágenes y etiquetas en un conjunto general. A partir de este conjunto general, extrae aleatoriamente las imágenes y sus etiquetas para formar el conjunto de datos de entrenamiento. Después, si se ha especificado un porcentaje para la validación, el método extrae una porción de estas imágenes para formar el conjunto de validación.

Finalmente, el método devuelve cuatro listas: las rutas de las imágenes de entrenamiento y validación, junto con sus etiquetas respectivas. Este proceso garantiza que los datos estén bien distribuidos y que el conjunto de validación sea una muestra representativa del conjunto de entrenamiento.

5. **entrenamiento_red_neuronal()** [267-298]: Realiza un paso del proceso de entrenamiento de la red neuronal. Durante este proceso, se toma un lote de imágenes y sus etiquetas correspondientes del conjunto de datos de entrenamiento. Estas imágenes y etiquetas se transfieren al dispositivo adecuado

(como una GPU si está disponible) para que el entrenamiento sea más eficiente.

La red neuronal se coloca en modo de entrenamiento, lo que le permite ajustar sus parámetros. Luego, el optimizador de la red se reinicia para asegurarse de que los gradientes anteriores no interfieran con el nuevo cálculo. La red neuronal genera predicciones para el lote de imágenes, que se comparan con las etiquetas reales para calcular la pérdida, que mide la diferencia entre las predicciones y los valores reales.

Esta pérdida se retropropaga a través de la red, permitiendo que el optimizador ajuste los pesos de las conexiones dentro de la red para mejorar su rendimiento en futuras predicciones. La pérdida del lote se acumula para calcular el rendimiento promedio durante el entrenamiento, y se incrementa un contador que lleva el seguimiento del número de imágenes procesadas. Este proceso se repite para cada lote de imágenes hasta completar el conjunto de entrenamiento.

6. **Método de validación de la red neuronal [302-328]:** Evalúa el rendimiento de la red neuronal en un conjunto de datos que no se utiliza para el entrenamiento, conocido como conjunto de validación. Este método es crucial para comprobar cómo generaliza la red a nuevos datos, evitando así que el modelo se ajuste demasiado a los datos de entrenamiento.

En este proceso, se toma un lote de imágenes y sus etiquetas correspondientes del conjunto de validación. Estas imágenes y etiquetas se transfieren al dispositivo adecuado (como una GPU, si está disponible). La red neuronal se coloca en modo de evaluación, lo que desactiva características como el dropout y evita que los gradientes se calculen, dado que en esta fase no se desea modificar los pesos de la red.

La red genera predicciones para las imágenes del lote, y estas predicciones se comparan con las etiquetas reales para calcular la pérdida. La pérdida obtenida se acumula para calcular la pérdida promedio durante la validación. Finalmente, se incrementa un contador que lleva el seguimiento del número de imágenes

procesadas durante la validación.

Este proceso permite medir el rendimiento de la red en datos no vistos previamente, ayudando a determinar si el modelo está generalizando bien o si, por el contrario, está sobreajustado a los datos de entrenamiento.

7. **Método de generación de resultados [343-391]:** Se encarga de compilar y guardar los resultados finales del proceso de entrenamiento de la red neuronal una vez que todas las épocas han sido completadas. Este método realiza varias tareas importantes para documentar y visualizar el rendimiento del modelo.

Primero, guarda el estado final de la red neuronal en un archivo en la ruta de destino especificada como se muestra en la figura 3.16. Esto permite que el modelo entrenado pueda ser reutilizado o evaluado más adelante sin necesidad de repetir el proceso de entrenamiento.

Luego, organiza los resultados de cada época, incluyendo las pérdidas promedio obtenidas tanto en el conjunto de entrenamiento como en el de validación. Estos resultados se estructuran en un formato legible, almacenándose en un archivo JSON como muestra la figura 3.17. Este archivo sirve como un registro detallado de los parámetros de entrenamiento y los resultados obtenidos, facilitando el análisis y la comparación con futuros entrenamientos.

Además, el método genera una gráfica que muestra la evolución de las pérdidas a lo largo de las épocas para el conjunto de entrenamiento y validación. Esta gráfica es una herramienta visual clave para entender cómo ha mejorado (o no) el modelo con el tiempo y si ha habido problemas como el sobreajuste. La gráfica se guarda como una imagen en la ruta de destino.

Finalmente, el método imprime un resumen de los resultados y la configuración del entrenamiento en la consola, proporcionando un acceso rápido a esta información para su revisión inmediata. Esto incluye detalles como el número total de épocas, la tasa de aprendizaje, el tamaño del lote, el número de imágenes utilizadas, y la ubicación donde se guardaron los resultados y el modelo.

Este proceso asegura que toda la información relevante del entrenamiento esté documentada y disponible para análisis posteriores.

Este desarrollo de código abarca la estructura y funcionalidad del programa, desde la carga del dataset y creación de red neuronal hasta su entrenamiento y validación y posterior guardado en el disco, proporcionando un flujo de trabajo eficiente y robusto para el entrenamiento de redes neuronales con la librería PyTorch.

3.4.6. Resultados

Durante esta fase del proyecto, se desarrolla un código que lleva a cabo el entrenamiento de una red neuronal y genera varios archivos importantes como resultado de este proceso.

El primero de estos archivos es un archivo con extensión “*.pth*”, que contiene los parámetros entrenados de la red neuronal como se muestra en la figura 3.16. Este archivo es esencial para almacenar el estado del modelo después del entrenamiento, permitiendo su uso posterior para inferencias o para continuar entrenando a partir de ese punto.

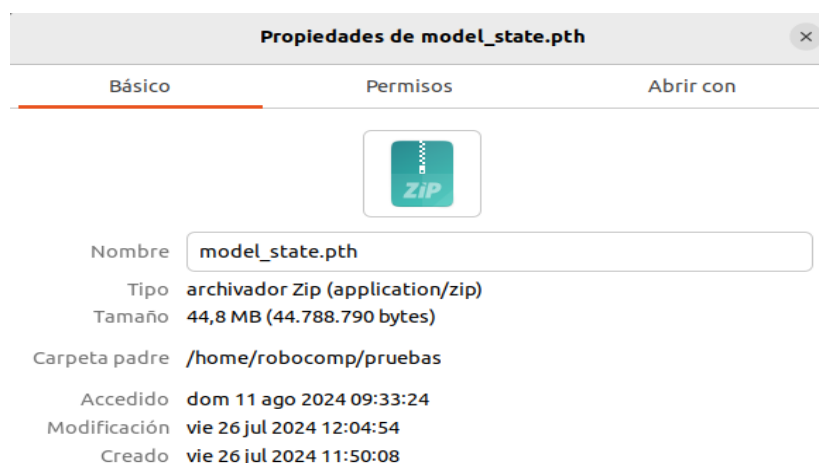


Figura 3.16: Archivo que guarda los parámetros entrenados de la red.

Además, el código genera un gráfico que visualiza el progreso del entrenamiento como se muestra en la figura 3.17, mostrando la variación de las pérdidas tanto en el conjunto de entrenamiento como en el de validación a lo largo de cada época. Este

3.4. FASE 4. ENTRENAMIENTO DE LA RED NEURONAL

gráfico es crucial para analizar el comportamiento del modelo durante el entrenamiento y ajustar los hiperparámetros según sea necesario.

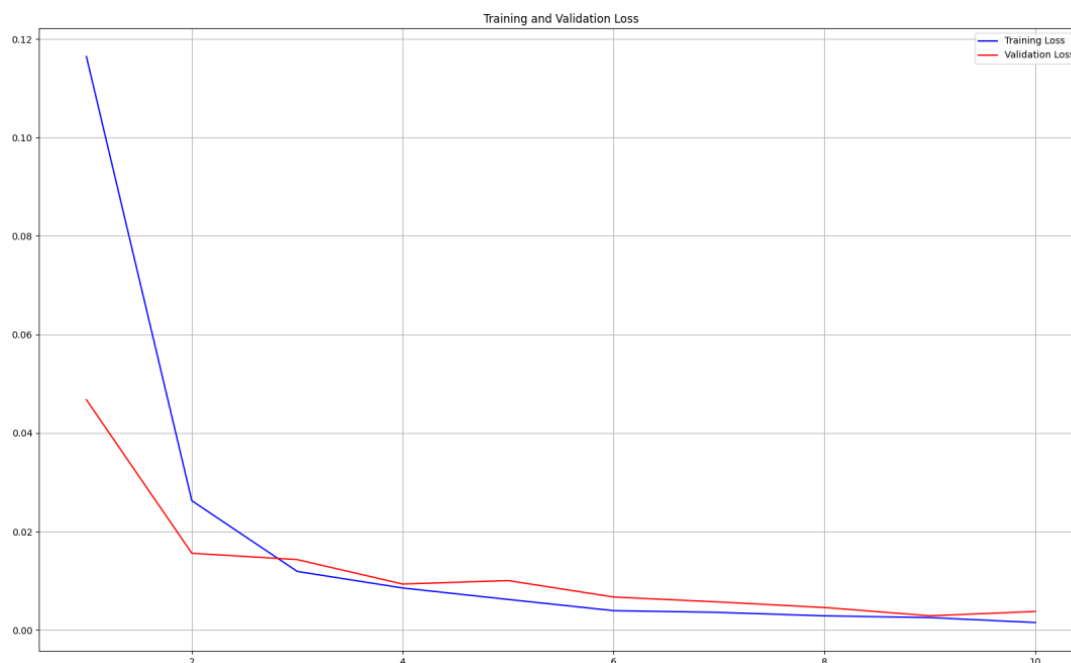


Figura 3.17: Gráfico de los resultados de entrenamiento y validación.

Finalmente, el código también crea un archivo JSON, como el que se muestra en la figura 3.18, en el que se guardan tanto los parámetros utilizados en el entrenamiento como los resultados obtenidos durante las fases de entrenamiento y validación. Además de almacenarse en este archivo, la información se muestra por consola para una revisión inmediata. Este archivo JSON proporciona un registro detallado y estructurado de todo el proceso, facilitando su análisis y documentación.

Como se puede apreciar en la figura 3.18 el tiempo medio de procesamiento de un lote de datos es de 0.3 segundos y el tiempo total de ejecución del código es de unos 483 segundos. A su vez la pérdida final alcanzada es de 0.001 lo que implicaría un 0.1 % de pérdida (Falla 1 predicción cada 1000).

```

1{"Ruta destino del modelo": "/home/robocomp/model state.pth",
2 "Numero de epocas": 10, "Tasa de aprendizaje": 0.001,
3 "Tamano de batch": 32,
4 "Resultados entrenamiento y validacion": [
5   "EPOCA[1/10] - Perdida Entrenamiento: 0.13740145 - Perdida validacion: 0.07769441 - Tiempo Medio: 0.00477208",
6   "EPOCA[2/10] - Perdida Entrenamiento: 0.033868 - Perdida validacion: 0.02121833 - Tiempo Medio: 0.00349336",
7   "EPOCA[3/10] - Perdida Entrenamiento: 0.01511354 - Perdida validacion: 0.01353295 - Tiempo Medio: 0.003469",
8   "EPOCA[4/10] - Perdida Entrenamiento: 0.00934087 - Perdida validacion: 0.0135544 - Tiempo Medio: 0.00356234",
9   "EPOCA[5/10] - Perdida Entrenamiento: 0.00728122 - Perdida validacion: 0.01233152 - Tiempo Medio: 0.00347908",
10  "EPOCA[6/10] - Perdida Entrenamiento: 0.00558917 - Perdida validacion: 0.00758113 - Tiempo Medio: 0.00351519",
11  "EPOCA[7/10] - Perdida Entrenamiento: 0.00378313 - Perdida validacion: 0.00622439 - Tiempo Medio: 0.00352454",
12  "EPOCA[8/10] - Perdida Entrenamiento: 0.00335082 - Perdida validacion: 0.00348412 - Tiempo Medio: 0.00333939",
13  "EPOCA[9/10] - Perdida Entrenamiento: 0.00344581 - Perdida validacion: 0.00177803 - Tiempo Medio: 0.00355372",
14  "EPOCA[10/10] - Perdida Entrenamiento: 0.00166948 - Perdida validacion: 0.00183213 - Tiempo Medio: 0.00349231"
15 ],
16 "Tiempo medio": "483.4312217235565"
17 }

```

Figura 3.18: Ejemplo del archivo JSON generado.

3.4.7. Conclusiones

Los resultados obtenidos durante la fase de entrenamiento y validación de la red neuronal son válidos y sirven para los propósitos prácticos del proyecto. Se observa una clara tendencia a la disminución del valor de las pérdidas tanto en el conjunto de entrenamiento como en el de validación, lo que indica que el modelo está aprendiendo pero no que este sea valido ya que para esto se necesitaría otro tipo de métricas como precisión.

Aunque se notan ligeras subidas en los valores de pérdida en ciertas épocas, estos incrementos son mínimos y ocurren con poca frecuencia. Estas variaciones no representan un problema significativo en el proceso de entrenamiento o validación. De hecho, son comunes y esperadas en muchos procesos de optimización debido a la naturaleza estocástica de los algoritmos de entrenamiento y la posible variabilidad en los datos de los lotes.

A pesar de que los resultados obtenidos son satisfactorios y útiles para la práctica, siempre hay espacio para mejoras. La optimización de hiperparámetros, la adquisición de más datos de entrenamiento, y el ajuste de la arquitectura de la red son algunas de las áreas donde se podrían realizar ajustes para mejorar aún más el rendimiento del modelo.

En conclusión, la fase de entrenamiento y validación ha sido exitosa, produciendo resultados que cumplen con los objetivos del proyecto. Las salidas obtenidas proporcionan una base sólida sobre la cual se puede seguir construyendo y optimizando el modelo en futuras iteraciones.

3.4.8. Ampliaciones a futuro

Para mejorar y optimizar el proyecto, se pueden considerar las siguientes ampliaciones futuras:

- ★ **Mejora de la comprobación de requisitos mínimos:** Implementar una verificación exhaustiva de todos los requisitos antes de iniciar la ejecución. Esto incluiría asegurar que todas las rutas de archivos y directorios existen, que las configuraciones de hardware y software son adecuadas, y que los parámetros de entrada cumplen con los límites y expectativas establecidos. Esta comprobación garantizaría que el entorno esté completamente preparado para el entrenamiento de la red neuronal, reduciendo la posibilidad de errores durante la ejecución.
- ★ **Mejorar el proceso de entrenamiento de la red:** Se puede explorar la optimización de varios aspectos del proceso de entrenamiento y validación para lograr un menor valor de pérdidas. Esto podría incluir la experimentación con diferentes arquitecturas de redes neuronales y la búsqueda de hiperparámetros óptimos, como el tamaño del lote (batch size) y la tasa de aprendizaje (learning rate). También se pueden probar técnicas de regularización adicionales, como la normalización por lotes (batch normalization) y el abandono (dropout), para mejorar la capacidad de generalización del modelo.
- ★ **Implementar paradas forzadas en el proceso de entrenamiento:** Desarrollar mecanismos para interrumpir el entrenamiento si se detectan comportamientos preocupantes, como subidas drásticas en el valor de pérdida o varias épocas consecutivas con pérdidas en aumento en lugar de disminución. Además, se podría implementar una estrategia de ajuste dinámico de la tasa de aprendizaje (learning rate) en función del rendimiento del modelo durante el entrenamiento. Esto permitiría reducir la tasa de aprendizaje si el modelo deja de mejorar, ayudando a evitar la oscilación y convergiendo más rápidamente a un valor de pérdida óptimo.

CAPÍTULO 3. SOLUCIÓN PROPUESTA

Estas ampliaciones futuras no solo aumentarían la robustez del proceso de entrenamiento y validación, sino que también mejorarían la calidad del modelo resultante, haciéndolo más eficiente y fiable para aplicaciones prácticas.



3.4. FASE 4. ENTRENAMIENTO DE LA RED NEURONAL

Capítulo 4

Implementación del sistema final

4.1. Introducción

En esta última fase del proyecto, nos enfocaremos en la implementación final del sistema, integrando y optimizando los componentes desarrollados en las fases previas para asegurar un funcionamiento robusto y en tiempo real. El objetivo principal de esta fase es lograr la detección, identificación y seguimiento de la persona objetivo dentro de un entorno dinámico utilizando tecnologías avanzadas de visión por computadora e inteligencia artificial.

El sistema se basa en una conexión continua con una cámara compatible con la biblioteca `pyrealsense2`, que proporciona una transmisión de fotogramas en tiempo real. Estos fotogramas son procesados periódicamente por el código, permitiendo una evaluación constante del entorno. A partir de cada fotograma, se aplica la red neuronal YOLO, una de las más eficientes para la identificación de regiones de interés (ROIs), especialmente en la detección de personas. Esta red permite la segmentación precisa y rápida de las áreas donde se encuentran los individuos en la escena.

Una vez que se han identificado las ROIs correspondientes a personas, el siguiente paso consiste en analizar estas regiones para determinar si alguna de ellas corresponde a la persona objetivo. Para ello, se emplea una red neuronal previamente entrenada durante la fase anterior del proyecto, que ha sido optimizada para evaluar similitudes

4.2. OBJETIVOS

con la persona objetivo. Este modelo ofrece un porcentaje de similitud que permite seleccionar de manera precisa la caja de colisión correspondiente a dicha persona.

Esta fase no solo sintetiza el trabajo realizado en las etapas anteriores, sino que también lo lleva a un nivel de aplicabilidad práctica, permitiendo la operación del sistema en tiempo real y garantizando una identificación confiable de la persona objetivo en diversas condiciones del entorno. Con ello, se sienta la base para la aplicación final del sistema en situaciones del mundo real, donde la precisión y la velocidad son cruciales.

4.2. Objetivos

En esta fase del proyecto, se establecen una serie de objetivos claros y específicos que guiarán la implementación final del sistema para la detección e identificación en tiempo real de la persona objetivo dentro de un fotograma de video. Estos objetivos están diseñados para asegurar que el sistema funcione con alta precisión y eficiencia, integrando de manera efectiva las tecnologías desarrolladas en las fases anteriores. A continuación, se detallan los objetivos de esta fase:

- ★ **Conexión con la cámara RGBD:** Establecer una conexión estable y continua con la cámara RGBD para capturar fotogramas en tiempo real. Esto incluye asegurar que los datos de profundidad y color se reciban de manera sincronizada y puedan ser procesados eficientemente por el sistema.
- ★ **Obtención y análisis de las regiones de interés:** Procesar los fotogramas capturados para detectar y extraer las áreas relevantes donde se encuentran personas en la escena. Este análisis permite identificar las regiones que serán examinadas más a fondo para determinar si contienen a la persona objetivo.
- ★ **Identificación de persona objetivo:** Evaluar las regiones de interés previamente identificadas para determinar si alguna corresponde a la persona objetivo. Esto se logra utilizando la red neuronal entrenada en fases anteriores, la cual compara

las características visuales de las personas detectadas con un perfil previamente definido, garantizando así la selección precisa de la persona deseada.

- ★ **Mostrar resultados del proceso:** Visualizar en tiempo real los resultados obtenidos, incluyendo la detección de personas y la identificación de la persona objetivo. Este objetivo también incluye la presentación de información relevante más intuitivamente asignando colores (Verde para el objetivo y rojo para el resto).

Este desarrollo de código abarca la estructura y funcionalidad del programa, desde la conexión con la cámara y la carga de los parámetros del modelo y posterior procesamiento de imágenes con dicha red, proporcionando un flujo de trabajo eficiente y robusto.

4.3. Explicaciones técnicas

Se aborda el funcionamiento interno y los procesos detallados que subyacen a las tecnologías y métodos mencionados. Esta sección proporciona una comprensión profunda de cómo se implementan y operan las técnicas, destacando los principios fundamentales. A través de estas explicaciones, se busca ofrecer una visión clara y detallada de cómo se llevan a cabo las tareas técnicas, permitiendo apreciar la complejidad y la eficiencia de las soluciones aplicadas.

- ★ **Red neuronal YOLOv8s:** Es un modelo de visión por computadora para la detección de objetos en tiempo real. Funciona dividiendo la imagen en una cuadrícula y aplicando una red neuronal convolucional para predecir simultáneamente las coordenadas de los cuadros delimitadores, las clases de objetos, y las probabilidades de esos objetos en cada celda de la cuadrícula. A diferencia de otros métodos, YOLO realiza todas estas predicciones en una sola pasada de la imagen, lo que lo hace extremadamente rápido y eficiente.

4.4. LIBRERÍAS PRINCIPALES

- ★ **Conexión con la cámara:** Se realiza configurando un pipeline que gestiona el flujo de datos en tiempo real. Este pipeline se inicializa para capturar fotogramas de flujos específicos, como el de color o profundidad. Para evitar bloqueos durante la solicitud de un fotograma, se utiliza un temporizador que abandona la solicitud si no se obtiene uno en un tiempo predefinido, devolviendo un valor nulo si se supera este límite. Al finalizar la captura, se detiene el pipeline para liberar los recursos de la cámara.
- ★ **Explicación de cajas de colisiones:** Divide la imagen en una cuadrícula y, para cada celda de la cuadrícula, predice múltiples cajas de colisión, cada una con coordenadas que definen su posición (centro, ancho, alto), la clase del objeto y la probabilidad de que un objeto de esa clase esté presente. Estas cajas son refinadas mediante un proceso de supresión de no-máximos (NMS), que elimina cajas redundantes y mantiene solo las más precisas y relevantes. Así, cada caja de colisión final refleja la ubicación y la identidad del objeto detectado con un alto grado de precisión.

4.4. Librerías principales

Este apartado examina las herramientas y bibliotecas esenciales que son fundamentales para el desarrollo de soluciones tecnológicas. Aquí se destacan las librerías más relevantes, describiendo sus funciones clave y cómo contribuyen a simplificar y mejorar el proceso de desarrollo. Este análisis permite comprender el papel de cada librería en la optimización y eficiencia de las aplicaciones tecnológicas.

- ★ **PyRealSense2[10]:** Proporciona una interfaz de Python para interactuar con las cámaras Intel RealSense. Permite capturar datos de imágenes en color, profundidad y otras métricas, facilitando la configuración, el control y la obtención de fotogramas de las cámaras en tiempo real. Con pyrealsense2, los desarrolladores pueden acceder a funcionalidades avanzadas, como la calibración de la cámara y la sincronización de múltiples sensores, simplificando

el proceso de integración y manejo de estas cámaras en aplicaciones de visión por computadora.

```
# Instalación de la librería PyRealSense2  
pip3 install pyrealsense2
```

- ★ **Ultralytics (YOLO)[16]:** Proporciona una implementación optimizada de los modelos YOLO para la detección de objetos. Su enfoque principal es ofrecer una versión mejorada y fácil de usar de YOLO, con soporte para entrenamiento, inferencia y evaluación de modelos de detección de objetos en tiempo real. La librería incluye versiones actualizadas de YOLO, como YOLOv8, que incorporan mejoras en precisión y velocidad. Además, facilita la integración con frameworks de aprendizaje profundo, y ofrece herramientas para la gestión de datos, la visualización de resultados y la personalización de modelos.

```
# Instalación de la librería Ultralytics  
pip3 install ultralytics
```

- ★ **PyTorch[19]:** Es un framework de código abierto para el aprendizaje profundo y la computación numérica. Ofrece una interfaz flexible y eficiente para construir y entrenar redes neuronales. Su enfoque en la programación dinámica de grafos permite realizar cálculos y modificaciones en tiempo real durante el entrenamiento, facilitando la experimentación y el desarrollo de modelos complejos. PyTorch es ampliamente utilizada en investigación y desarrollo debido a su facilidad de uso, rendimiento y compatibilidad con una amplia gama de herramientas y bibliotecas de machine learning.

```
# Instalación de la librería PyTorch  
pip3 install torch torchvision
```

- ★ **OpenCV[11]:** Es una biblioteca de código abierto para el procesamiento

de imágenes y visión por computadora. Proporciona herramientas extensivas para tareas como la captura y manipulación de imágenes, detección de objetos, reconocimiento facial, y análisis de video en tiempo real. OpenCV soporta múltiples lenguajes de programación y plataformas, facilitando la implementación de soluciones de visión por computadora y el desarrollo de aplicaciones que requieren procesamiento y análisis visual avanzados.

```
# Instalación de la librería OpenCV
pip3 install opencv-python
```

4.5. Desarrollo del código

4.5.1. Detección sobre vídeo pregrabado

En este apartado, se explicarán los métodos clave de la clase que emplea una red neuronal previamente entrenada para detectar a la persona objetivo en los fotogramas capturados por una cámara en tiempo real. Se detallará cómo se carga el modelo entrenado, cómo se procesa cada fotograma de la grabación y cómo se identifican y resaltan las personas objetivo entre el resto. Se puede consultar la implementación completa de la clase en el repositorio de GitHub[22].

1. **compute() [104-126]:** Gestiona el flujo de trabajo para procesar imágenes de una fuente de grabación. Primero, intenta recibir un fotograma de la grabación. Si el fotograma se recibe con éxito, se convierte en un formato adecuado para su análisis. Luego, la imagen se procesa para detectar objetos, específicamente personas, usando una red neuronal preentrenada. A continuación, se extraen y filtran los resultados relevantes, como las posiciones de las personas detectadas. Se selecciona la persona objetivo basándose en predicciones realizadas por otra red neuronal, y se dibujan rectángulos alrededor de las personas en la imagen original. Finalmente, se muestran dos ventanas con la imagen original y la

imagen con los resultados de detección. Si no se recibe un fotograma, el método termina la ejecución con un mensaje de error.

2. **comprobacion_requisitos_minimos()** [130-138]: Verifica que los archivos y directorios necesarios para la ejecución del programa existen y están accesibles. Primero, comprueba si la ruta del archivo de grabación está presente en el sistema; si no es así, finaliza el proceso con un mensaje de error. Luego, realiza una verificación similar para la ruta de los parámetros de la red neuronal. Si cualquiera de estas rutas no existe, también se termina la ejecución con un mensaje de error. Este método asegura que todos los recursos necesarios están disponibles antes de proceder con la inicialización del sistema.
3. **iniciar_conexion_grabacion()** [142-153]: Establece la conexión con el dispositivo de captura de video y configura la lectura de datos desde un archivo. Primero, se crea un objeto para la conexión y configuración del dispositivo de grabación. Luego, se configura el dispositivo para que lea desde el archivo especificado en la ruta de grabación, con la opción de no repetir la reproducción del archivo. Finalmente, se inicia la conexión utilizando esta configuración, permitiendo la captura de datos del archivo de video para su procesamiento posterior.
4. **preparacion_entorno()** [157-166]: Se encarga de configurar los recursos necesarios para el procesamiento de imágenes y la detección de objetos. Primero, carga el modelo de red neuronal para la detección de objetos en imágenes, específicamente un modelo de YOLO previamente entrenado. Luego, inicializa y carga una segunda red neuronal, diseñada para el seguimiento de objetivos, utilizando los parámetros almacenados en un archivo. Esta red se configura para que se ejecute en el dispositivo adecuado, ya sea una GPU o una CPU, según las capacidades del sistema.
5. **separacion_filtracion_resultados()** [184-197]: Filtra y procesa los resultados de detección de objetos para extraer información relevante. Comienza creando una

4.5. DESARROLLO DEL CÓDIGO

lista vacía para almacenar los datos de las detecciones. Luego, itera a través de los resultados de detección, verificando si cada objeto detectado es una persona y si la confianza en la detección supera un umbral mínimo. Si ambos criterios se cumplen, agrega las coordenadas de la caja delimitadora (bounding box) de la persona a la lista. Al final, devuelve la lista de cajas de colisión correspondientes a las personas detectadas.

6. **obtencion_indice_persona_objetivo()** [201-226]: Se encarga de identificar la persona objetivo entre las detecciones realizadas. Comienza inicializando variables para llevar el seguimiento del índice y valor máximo de la predicción. Para cada caja delimitadora que corresponde a una persona, extrae la región de interés (ROI) de la imagen original. Esta región se prepara para la red neuronal transformándola en un tensor y moviéndola al dispositivo adecuado. Luego, la red neuronal evalúa la ROI para obtener una predicción de relevancia. Si la predicción es mayor que el valor máximo registrado y supera un umbral mínimo, actualiza el índice y el valor máximo. Finalmente, devuelve el índice de la persona con la mayor predicción relevante.
7. **aplicar_resultados_fotograma()** [230-252]: Se encarga de visualizar los resultados de las detecciones sobre el fotograma original. Comienza haciendo una copia del fotograma para trabajar con ella. Luego, dibuja rectángulos alrededor de las áreas detectadas en la imagen original. Utiliza un color verde para el rectángulo de la persona que ha sido identificada como objetivo y rojo para las demás personas. Cada rectángulo se dibuja en las coordenadas especificadas por las cajas delimitadoras. Finalmente, devuelve la imagen con los resultados superpuestos, mostrando las detecciones y destacando al objetivo.

4.5.2. Detección sobre video en tiempo real

En este apartado, se explicarán los métodos clave de la clase que utiliza una red neuronal previamente entrenada para detectar a la persona objetivo en tiempo real sobre los fotogramas. Se detalla cómo se carga el modelo entrenado, cómo se procesa cada fotograma en vivo y cómo se identifican y destacan las personas objetivo entre el resto. Se puede consultar la implementación completa de la clase en el repositorio de GitHub[23].

1. **compute() [106-128]:** Captura fotogramas de la cámara y verifica si se ha recibido una imagen válida. Si es así, convierte el fotograma a un formato adecuado y aplica un modelo para detectar personas en la imagen. Luego filtra las detecciones para conservar solo aquellas con alta precisión y extrae las coordenadas de las personas detectadas. A continuación, evalúa cada persona detectada usando una red neuronal para identificar cuál es el objetivo de seguimiento, basándose en la puntuación más alta y un umbral de precisión. Marca a la persona objetivo en la imagen con un rectángulo verde, mientras que otras personas se destacan con rectángulos rojos. Finalmente, muestra las imágenes procesadas en la interfaz de usuario para que el usuario pueda ver tanto la imagen original como la con los resultados de la detección y seguimiento.
2. **comprobacion_requisitos_minimos() [132-136]:** Verifica la existencia de ciertos archivos necesarios para la operación del sistema. Primero, comprueba si el archivo de parámetros de la red neuronal, ubicado en una ruta específica, existe. Si el archivo no está presente, el método termina el programa e informa de un error, indicando que la ruta de los parámetros de la red neuronal es incorrecta. Esta verificación asegura que todos los recursos necesarios para la correcta configuración y funcionamiento del sistema están disponibles antes de continuar con el proceso.
3. **iniciar_conexion_cámara() [140-166]:** Establece una conexión con una cámara para capturar imágenes y datos. Inicialmente, se crean los objetos necesarios

4.5. DESARROLLO DEL CÓDIGO

para la conexión y configuración de la cámara. Se especifica el número de serie de la cámara para identificar el dispositivo. Luego, se configuran los flujos de datos, activando la transmisión de imágenes en color y de profundidad, con las resoluciones y tasas de fotogramas especificadas. Finalmente, se inicia la conexión con la cámara utilizando esta configuración, permitiendo la captura continua de imágenes y datos desde el dispositivo.

4. **preparacion_entorno() [170-179]:** Se encarga de configurar los recursos necesarios para el procesamiento de imágenes y la detección de objetos. Primero, carga el modelo de red neuronal para la detección de objetos en imágenes, específicamente un modelo de YOLO previamente entrenado. Luego, inicializa y carga una segunda red neuronal, diseñada para el seguimiento de objetivos, utilizando los parámetros almacenados en un archivo. Esta red se configura para que se ejecute en el dispositivo adecuado, ya sea una GPU o una CPU[24], según las capacidades del sistema.
5. **separacion_filtracion_resultados() [197-210]:** Filtra y procesa los resultados de detección de objetos para extraer información relevante. Comienza creando una lista vacía para almacenar los datos de las detecciones. Luego, itera a través de los resultados de detección, verificando si cada objeto detectado es una persona y si la confianza en la detección supera un umbral mínimo. Si ambos criterios se cumplen, agrega las coordenadas de la caja delimitadora (bounding box) de la persona a la lista. Al final, devuelve la lista de cajas de colisión correspondientes a las personas detectadas.
6. **obtencion_indice_persona_objetivo() [214-239]:** Se encarga de identificar la persona objetivo entre las detecciones realizadas. Comienza inicializando variables para llevar el seguimiento del índice y valor máximo de la predicción. Para cada caja delimitadora que corresponde a una persona, extrae la región de interés (ROI) de la imagen original. Esta región se prepara para la red neuronal transformándola en un tensor y moviéndola al dispositivo adecuado. Luego,

la red neuronal evalúa la ROI para obtener una predicción de relevancia. Si la predicción es mayor que el valor máximo registrado y supera un umbral mínimo, actualiza el índice y el valor máximo. Finalmente, devuelve el índice de la persona con la mayor predicción relevante.

7. **aplicar_resultados_fotograma()** [243-265]: Se encarga de visualizar los resultados de las detecciones sobre el fotograma original como se muestra en la figura 4.1. Comienza haciendo una copia del fotograma para trabajar con ella. Luego, dibuja rectángulos alrededor de las áreas detectadas en la imagen original. Utiliza un color verde para el rectángulo de la persona que ha sido identificada como objetivo y rojo para las demás personas. Cada rectángulo se dibuja en las coordenadas especificadas por las cajas delimitadoras. Finalmente, devuelve la imagen con los resultados superpuestos, mostrando las detecciones y destacando al objetivo.

4.6. Resultados

El sistema desarrollado ofrece una visualización en tiempo real de dos fotogramas clave: el fotograma original capturado por la cámara RealSense y un fotograma procesado que muestra los resultados de la detección y seguimiento de personas. El fotograma original presenta la imagen tal como es capturada por la cámara, sirviendo como referencia base. En el fotograma procesado, se aplican bounding boxes para marcar a las personas detectadas. Estas se destacan con un código de colores predefinido: rojo para las personas que no son el objetivo principal de seguimiento y verde para la persona seleccionada como objetivo.

El sistema demuestra una alta precisión y consistencia en la detección. Utiliza la red neuronal YOLO para identificar personas en los fotogramas y, posteriormente, una red neuronal personalizada basada en ResNet para determinar cuál de estas personas será el objetivo principal. A lo largo de toda la ejecución del video, la red neuronal selecciona consistentemente al objetivo correcto sin errores, garantizando un seguimiento fiable y

4.7. CONCLUSIONES

estable.

En conjunto, el sistema no solo permite al usuario visualizar en tiempo real el proceso de seguimiento de personas, sino que lo hace con un alto grado de precisión y confiabilidad. Esto asegura que la persona objetivo se identifique y se destaque correctamente durante todo el video, lo que es crucial en aplicaciones como la vigilancia o la interacción de robots en entornos dinámicos.

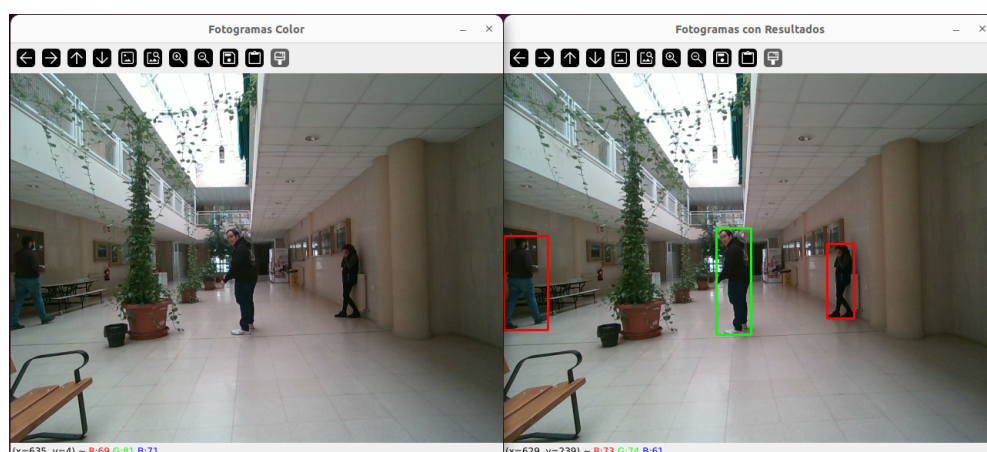


Figura 4.1: Interfaz de usuario con la imagen original y otra con los resultados.

4.7. Conclusiones

Los resultados obtenidos durante la implementación y pruebas finales del modelo preentrenado para la detección de la persona objetivo son altamente satisfactorios y cumplen con los propósitos prácticos del proyecto. El modelo muestra una precisión muy alta, con un margen de error aproximado de solo 0.001, lo que indica un desempeño sólido en la tarea de identificar y seguir a la persona objetivo. A pesar de la precisión alcanzada, el modelo mantiene un comportamiento estable y no experimenta cambios drásticos en su desempeño, incluso en condiciones variables.

En cuanto a la capacidad de generalización del modelo, se observa un desempeño excelente cuando se trata de detectar a la persona objetivo, independientemente de los nuevos fotogramas o entornos diferentes a los usados durante el entrenamiento. Sin

embargo, en lo que respecta a la detección de personas que no son el objetivo principal, el modelo muestra una precisión menor, lo cual sugiere un área de mejora.

El seguimiento del objetivo es consistente cuando el modelo se aplica al video sobre el que fue entrenado, sin presentar fallos ni errores de seguimiento. Esto refuerza la eficacia del modelo dentro del contexto específico para el cual fue diseñado.

A pesar de los buenos resultados obtenidos, hay margen para mejorar el modelo. Una de las mejoras clave sería aumentar el tamaño del dataset, especialmente incrementando el número de personas distintas al objetivo en los datos de entrenamiento. Actualmente, el modelo ha sido entrenado con un número limitado de personas no objetivo, lo que podría limitar su capacidad para generalizar en escenarios con mayor diversidad de individuos.

En conclusión, la fase final de implementación ha sido exitosa, logrando resultados que cumplen con los objetivos del proyecto. El modelo muestra un rendimiento robusto y fiable, pero también presenta oportunidades de optimización que podrían explorarse en futuras iteraciones para mejorar aún más su precisión y capacidad de generalización.

4.8. Ampliaciones a futuro

En futuras iteraciones del proyecto, se identifican varias áreas clave que podrían ser ampliadas o mejoradas para aumentar la robustez y eficiencia del sistema:

- ★ **Mejora de la comprobación de requisitos mínimo:** Es fundamental reforzar el proceso de verificación de los requisitos mínimos necesarios para la ejecución del sistema. Esto implicaría una comprobación más exhaustiva de las dependencias, recursos de hardware disponibles (como la GPU), y otros factores críticos antes de que el sistema comience su operación. Implementar una validación automática que asegure que todas las condiciones necesarias están cumplidas podría prevenir errores y mejorar la estabilidad desde el inicio.
- ★ **Mejora del control de errores:** El sistema podría beneficiarse de un control de errores más robusto, especialmente en situaciones donde no se reciban

4.8. AMPLIACIONES A FUTURO

fotogramas de manera continua. Por ejemplo, implementar un mecanismo que detenga la ejecución o que intente reconectar automáticamente tras un número determinado de fotogramas perdidos aumentaría la resiliencia del sistema y evitaría fallos inesperados durante la operación.

- ★ **Mejorar la red neuronal:** Se plantea la necesidad de optimizar tanto la arquitectura de la red neuronal como el proceso de entrenamiento. Esto incluye explorar nuevas configuraciones de la red que puedan ofrecer mejor rendimiento y generalización, así como ampliar y diversificar el dataset de entrenamiento. Especialmente, incrementar el número de personas no objetivo en el dataset permitiría al modelo mejorar su precisión en escenarios más variados.
- ★ **Añadir el proceso de entrenamiento en tiempo real:** Una extensión importante sería implementar la capacidad de entrenar la red neuronal en tiempo real mientras está en operación. Esto implicaría almacenar los ROIs (Regiones de Interés) detectados durante la ejecución y, una vez alcanzado un tamaño de lote determinado, usar esos datos para entrenar el modelo en tiempo real. Aunque se intentó implementar esta funcionalidad, se encontró con errores técnicos que impidieron su finalización. Resolver estos problemas y lograr la integración del entrenamiento en tiempo real podría permitir que el modelo se adapte y mejore continuamente durante su uso.

Estas ampliaciones no solo mejorarían el rendimiento y la fiabilidad del sistema, sino que también permitirían que el modelo evolucione y se adapte de manera más efectiva a nuevas situaciones y entornos.

Capítulo 5

Resultados

Los resultados obtenidos durante el desarrollo del proyecto son una muestra clara de que se ha logrado implementar un sistema básico pero completamente funcional, que satisface los objetivos planteados. El código desarrollado cumple con las especificaciones requeridas para realizar el seguimiento de una persona específica y distinguirla del resto de los individuos presentes en la escena. Aunque es consciente que existen áreas donde la eficiencia podría mejorarse, el código actual proporciona una solución robusta y efectiva, que demuestra la viabilidad del enfoque adoptado.

El sistema es capaz de generar resultados detallados a partir de diversas configuraciones y procesos implementados. Esto incluye la configuración inicial de la cámara, donde se especifican parámetros como la resolución y la tasa de fotogramas, así como la configuración de las redes neuronales utilizadas, como las arquitecturas y los parámetros de entrenamiento. Además, el código ofrece información detallada sobre los resultados obtenidos durante la ejecución del proceso, como el número de imágenes guardadas y procesadas, y las métricas asociadas a cada etapa del entrenamiento y validación de la red neuronal.

Un aspecto clave de los resultados es la capacidad del sistema para almacenar archivos críticos en el disco. Esto incluye el guardado de los parámetros de la red neuronal después de que ha sido entrenada, lo que permite su reutilización o ajuste en futuras fases del proyecto. Además, el sistema guarda las grabaciones de

vídeo capturadas durante el proceso, lo que proporciona una referencia clara de las condiciones en las que se ha realizado el seguimiento. También se almacenan las imágenes utilizadas o generadas durante el proceso, lo que permite un análisis posterior más detallado y facilita la replicación del experimento en diferentes escenarios.

En conjunto, los resultados obtenidos no solo confirman que el sistema es capaz de cumplir con los objetivos iniciales, sino que también destacan su capacidad para generar y almacenar información valiosa que puede ser utilizada para mejorar y ajustar el modelo en futuras iteraciones. El código, aunque básico, ha demostrado ser una herramienta poderosa y flexible, capaz de adaptarse a las necesidades del proyecto y proporcionar resultados sólidos y confiables. Esto sienta una base sólida para futuras mejoras y optimizaciones que podrían incrementar tanto la eficiencia como la precisión del sistema.

Capítulo 6

Conclusiones

En conclusión, el proyecto ha logrado cumplir sus objetivos mediante la implementación de un código sencillo pero efectivo, capaz de realizar el seguimiento de una persona específica dentro de un entorno de manera confiable. A pesar de su simplicidad, el código se destaca por su diseño modular, lo que lo hace fácilmente ampliable y adaptable para futuras mejoras. Esta modularidad es un aspecto clave, ya que permite a los desarrolladores agregar nuevas funcionalidades, modificar componentes existentes, o adaptar el sistema a diferentes escenarios sin necesidad de reescribir grandes secciones del código. Esto facilita no solo el mantenimiento del sistema, sino también su evolución a medida que surjan nuevas necesidades o avances tecnológicos.

Aunque el sistema actual cumple con los requisitos funcionales del proyecto, la eficiencia es un área donde aún existe margen para mejoras. El código, en su forma actual, es suficientemente eficiente para los propósitos básicos del proyecto, pero no alcanza su máximo potencial en términos de rendimiento. Esto se debe en parte a que la mayor parte del procesamiento se realiza utilizando la CPU, lo que, si bien es aceptable, podría mejorarse significativamente mediante el uso de la GPU. La GPU está diseñada para manejar cálculos en paralelo y es particularmente eficaz en tareas como el procesamiento de imágenes y el entrenamiento de redes neuronales. Al aprovechar la GPU, el sistema podría procesar datos más rápidamente, manejar

conjuntos de datos más grandes, y realizar entrenamientos de redes neuronales de manera mucho más eficiente.

En resumen, este proyecto ha establecido una base sólida con un código que cumple con los objetivos planteados y que es fácilmente escalable y mejorable. La eficiencia actual es adecuada, pero se identifica un claro camino hacia la optimización mediante la implementación de procesamiento en GPU. Estas futuras mejoras podrían transformar el sistema en una herramienta aún más poderosa y rápida, capaz de enfrentar tareas más complejas y exigentes en el ámbito del seguimiento de personas y la inteligencia artificial aplicada a tareas cotidianas.

Capítulo 7

Trabajos Futuros

Para seguir avanzando en la evolución del sistema de seguimiento, se proponen varias ampliaciones y mejoras que pueden enriquecer la funcionalidad y eficacia del proyecto:

- ★ **Seguimiento de múltiples personas (Multi-Object Tracking, MOT[25]):**

Actualmente, el sistema está diseñado para seguir a una sola persona objetivo en una escena. Sin embargo, en entornos más complejos donde múltiples individuos están presentes, es crucial desarrollar capacidades de seguimiento de múltiples objetos. Implementar técnicas avanzadas de MOT permitirá al sistema identificar y seguir a varias personas simultáneamente, asignando identificadores únicos a cada una y gestionando la interacción entre ellos de manera efectiva. Este enfoque ampliará significativamente la aplicabilidad del sistema en escenarios más dinámicos y abarcará una gama más amplia de casos de uso.

- ★ **Generar un orden de prioridad para el seguimiento:** En situaciones donde no es posible seguir a todos los individuos simultáneamente debido a limitaciones de procesamiento o de la cámara, establecer un orden de prioridad para el seguimiento puede ser fundamental. Se podría implementar un sistema que asigne prioridades basadas en criterios específicos, como la proximidad al objetivo, el comportamiento o la importancia relativa en el contexto de la tarea. Esto permitirá al sistema optimizar los recursos y centrarse en los individuos

más relevantes según el criterio definido, mejorando así la eficacia general del seguimiento.

- ★ **Mejora en la gestión de la calidad del seguimiento[26]:** Para asegurar un seguimiento preciso y continuo, es importante mejorar la gestión de la calidad del seguimiento. Esto incluye la implementación de mecanismos para detectar y corregir errores en el seguimiento, como pérdidas temporales del objetivo debido a oclusiones o cambios en la apariencia. Se pueden incorporar técnicas de corrección y ajuste en tiempo real para mantener la precisión del seguimiento, así como herramientas de evaluación continua para monitorizar y ajustar la calidad del seguimiento durante su ejecución.
- ★ **Entrenamiento mientras sucede la detección:** Una de las áreas más innovadoras para explorar es la posibilidad de combinar la detección en tiempo real con el entrenamiento dinámico del modelo. Esta técnica implicaría almacenar los datos de detección en tiempo real y utilizarlos para actualizar el modelo de manera incremental. Por ejemplo, al guardar los ROI (Regiones de Interés) de cada detección y entrenar el modelo con estos datos en lotes, se puede adaptar el sistema a nuevas condiciones y mejorar su precisión con el tiempo. Aunque se intentó implementar este enfoque y se encontró con problemas técnicos, superar estos desafíos y perfeccionar la integración entre detección y entrenamiento podría llevar a un sistema más robusto y adaptable.

Estas ampliaciones no solo mejorarán la funcionalidad del sistema, sino que también ampliarán su aplicabilidad a una gama más amplia de escenarios y desafíos en el seguimiento de personas mediante inteligencia artificial. Cada uno de estos puntos representa una oportunidad para llevar el proyecto a un nivel superior, optimizando su rendimiento y adaptabilidad en entornos reales.

Bibliografía

- [1] Universidad de Extremadura RoboLab. Robolab website. <https://robolab.unex.es/>, 2024. Online; Accedido: 2024-08-27.
- [2] Microsoft. Vs code. <https://code.visualstudio.com/>, 2024. Online; Accedido: 2024-08-27.
- [3] OpenAI. Chatgpt: A large language model. <https://chatgpt.com/>, 2024. Online; Accedido: 2024-08-27.
- [4] GitHub. Github. <https://github.com/>, 2024. Online; Accedido: 2024-08-27.
- [5] Canonical Ltd. Ubuntu 22.04 lts. <https://releases.ubuntu.com/22.04/>, 2024. Online; Accedido: 2024-08-27.
- [6] Akeo Consulting. Rufus. <https://rufus.ie/es/>, 2024. Online; Accedido: 2024-08-27.
- [7] Balena. Ballenaetcher. <https://etcher.balena.io/>, 2024. Online; Accedido: 2024-08-27.
- [8] RoboLab. Robocomp: The robotic framework. <https://robocomp.github.io/web/>, 2024. Online; Accedido: 2024-08-27.
- [9] KDE. Yakuake terminal emulator. <https://apps.kde.org/ca/yakuake/>, 2024. Online; Accedido: 2024-08-27.

- [10] Intel. Pyrealsense2: Python api for intel realsense sdk. https://intelrealsense.github.io/librealsense/python_docs/_generated/pyrealsense2.html, 2024. Online; Accedido: 2024-08-27.
- [11] OpenCV Team. Opencv documentation. <https://docs.opencv.org/4.x/index.html>, 2024. Online; Accedido: 2024-08-27.
- [12] NumPy Developers. Numpy documentation. <https://numpy.org/doc/>, 2024. Online; Accedido: 2024-08-27.
- [13] Python Software Foundation. Json library documentation. <https://docs.python.org/3/library/json.html>, 2024. Online; Accedido: 2024-08-27.
- [14] Alejandro González Fernández. Código tfg: Video saver (fase 1). https://github.com/alejandroator19/codigo_tfg/blob/main/fase_1/video_saver/src/specificworker.py, 2024. Online; Accedido: 2024-08-27.
- [15] Zoumana Keita. Explicación funcionamiento yolo. <https://www.datacamp.com/es/blog/yolo-object-detection-explained>, 2024. Online; Accedido: 2024-08-27.
- [16] Ultralytics. Ultralytics documentation. <https://docs.ultralytics.com/>, 2024. Online; Accedido: 2024-08-27.
- [17] Alejandro González Fernández. Código tfg: Depth color frame saver (fase 2). https://github.com/alejandroator19/codigo_tfg/blob/main/fase_2/depth_color_frame_saver/src/specificworker.py, 2024. Online; Accedido: 2024-08-27.
- [18] Alejandro González Fernández. Código tfg: Dataset classifier (fase 2). https://github.com/alejandroator19/codigo_tfg/blob/main/fase_2/dataset_classifier/src/specificworker.py, 2024. Online; Accedido: 2024-08-27.

- [19] PyTorch. Pytorch documentation. <https://pytorch.org/docs/stable/index.html>, 2024. Online; Accedido: 2024-08-27.
- [20] Matplotlib. Matplotlib documentation. <https://matplotlib.org/stable/users/index.html>, 2024. Online; Accedido: 2024-08-27.
- [21] Alejandro González Fernández. Código tfg: Target detection trainer (fase 3). https://github.com/alejandroatori19/codigo_tfg/blob/main/fase_3/target_detection_trainer/src/specificworker.py, 2024. Online; Accedido: 2024-08-27.
- [22] Alejandro González Fernández. Código tfg: Video detección y selección de objetivos (fase 4). https://github.com/alejandroatori19/codigo_tfg/blob/main/fase_4/video_deteccion_and_selection_target/src/specificworker.py, 2024. Online; Accedido: 2024-08-27.
- [23] Alejandro González Fernández. Código tfg: Detección y selección de objetivos en tiempo real (fase 4). https://github.com/alejandroatori19/codigo_tfg/blob/main/fase_4/real_time_deteccion_and_selection_target/src/specificworker.py, 2024. Online; Accedido: 2024-08-27.
- [24] Dipesh Gyawali. Comparative analysis of cpu and gpu profiling for deep learning models. <https://arxiv.org/pdf/2309.02521>, 2023. Online; Accedido: 2024-08-27.
- [25] Zijia Lu. Self-supervised multi-object tracking with path consistency. <https://cvpr.thecvf.com/virtual/2024/poster/30783>, 2024. Online; Accedido: 2024-08-27.
- [26] Khan et al. A multiview approach to tracking people in crowded scenes. https://www.crcv.ucf.edu/papers/Khan_ECCV06.pdf, 2006. Online; Accedido: 2024-08-27.